

PREGRADO



GUÍA PARA REPOSITARIOS DE DOCS-AS-CODE

CONVENCIONES Y MEJORES PRÁCTICAS

FACULTAD DE INGENIERÍA | CARRERA DE INGENIERÍA DE SOFTWARE

INTRODUCCIÓN

Esta guía proporciona los conceptos fundamentales y recomendaciones para implementar un repositorio Git que aloja un informe de proyecto colaborativo.

La organización del repositorio propuesta pretende contener archivos Markdown modulares, GitFlow para el trabajo en equipo, Semantic Versioning, Conventional Commits y exportaciones profesionales a PDF mediante Pandoc.

AGENDA

01	MARKDOWN COMO CAPA DE ABSTRACCIÓN UNIVERSAL	<u>04</u>
02	GITFLOW WORKFLOW: ARQUITECTURA DE RAMAS Y CONTROL DE VERSIONES	<u>23</u>
03	VERSIONADO SEMÁNTICO 2.0.0 (SEMVER)	<u>38</u>
04	CONVENTIONAL COMMITS 1.0.0: CÓDIGO COMO COMUNICACIÓN	<u>53</u>
05	PANDOC: DE MARKDOWN A PDF A ESCALA	<u>68</u>
06	ARQUITECTURA DE UN REPOSITORIO GIT PARA REPORTES EN MARKDOWN	<u>83</u>

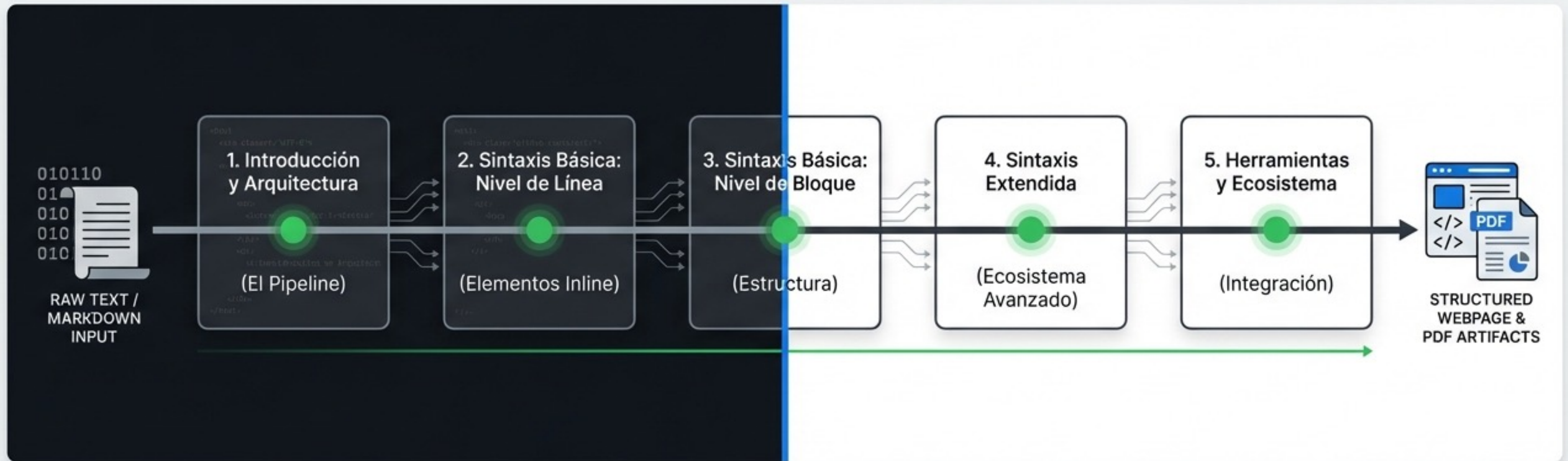
`title:` Universal Abstraction

`author:` Engineering Dept.

`# Markdown como Capa de Abstracción Universal`

Markdown como Capa de Abstracción Universal

Arquitectura y Especificación del Estándar



El Cambio de Paradigma en la Creación de Documentos

Editores WYSIWYG (Word/Docs)

Control de Versiones

Conflictos binarios opacos

Formato de Archivo

Bloqueo propietario (Lock-in)

Velocidad de Formato

Navegación por menús e interfaz gráfica

Longevidad (Future-Proof)

Depende del software comercial

Markdown

Control de Versiones

Diferencias línea por línea en Git

Formato de Archivo

Texto plano universal (Portable)

Velocidad de Formato

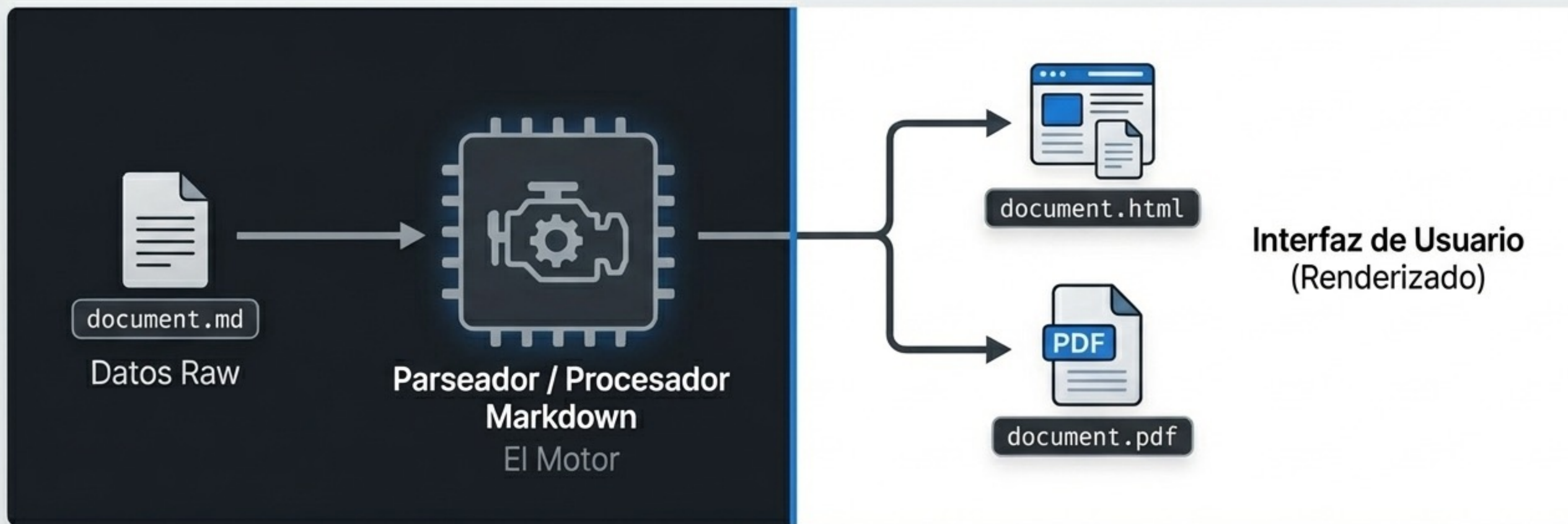
Manos en el teclado continuo

Longevidad (Future-Proof)

Legible indefinidamente

Nota Histórica: John Gruber creó Markdown en 2004 con el objetivo principal de la legibilidad humana en texto plano.

El Pipeline de Compilación de Datos a Presentación



Markdown aplica un principio central de la ingeniería de software: la estricta separación entre el contenido (datos) y su presentación (interfaz de usuario).

Ecosistema de Dialectos: Los “Sabores” de Markdown

	CommonMark (Estándar base)	GitHub Flavored Markdown (GFM)	MultiMarkdown
Sintaxis Básica	✓	✓	✓
Tablas	✗	✓	✓
Listas de Tareas	✗	✓	✗
Notas al pie	✗	✗	✓

Takeaway de la Sección 1

Markdown no es un estándar monolítico, sino un ecosistema de dialectos. El desarrollador debe conocer qué variante soporta la biblioteca o procesador que está utilizando.

Sintaxis a Nivel de Línea y la Lógica de Escape

`**bold text**`

`*italic text*`

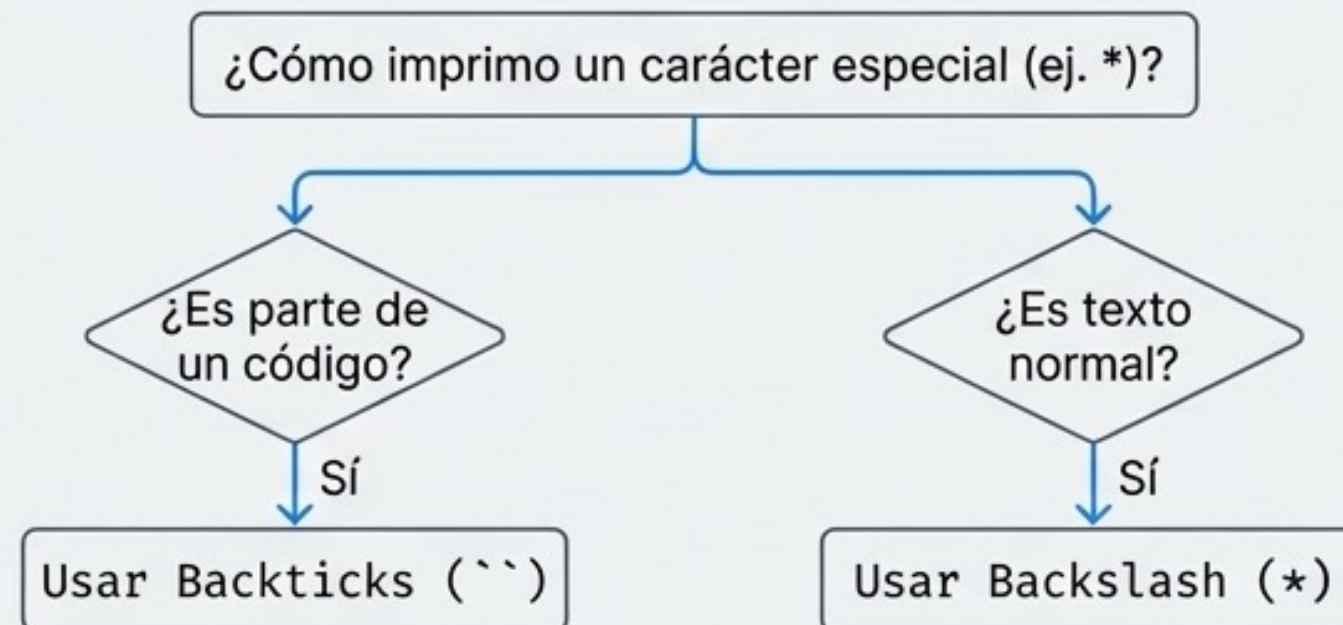
``inline code``

bold text

italic text

`inline code`

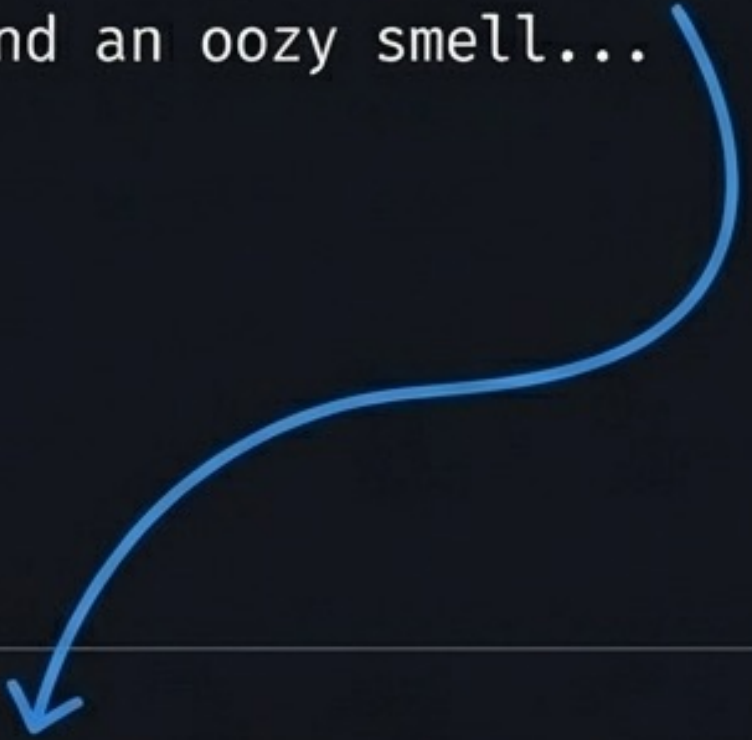
The Escaping Logic Tree



El carácter de escape universal (\) funciona para símbolos reservados como `_`, `{`, `[`, `#`, y `+`.

Topología de Enlaces por Referencia (Variables Estáticas)

In a hole in the ground there lived a [hobbit-hole][1]. Not a nasty, dirty, wet hole, filled with the ends of worms and an oozy smell...



[1]: <https://en.wikipedia.org/wiki/Hobbit#Lifestyle> "Hobbit lifestyles"

En lugar de saturar los párrafos con URLs largas, los enlaces por referencia actúan como punteros a variables estáticas definidas en el pie del documento, manteniendo el texto fuente limpio (principio DRY).

Inserción de Medios y Compatibilidad HTML

```
![alt text](image.jpg "Title")
```

Presiona `<kbd>Ctrl</kbd>` + `<kbd>C</kbd>`



Presiona `Ctrl` + `C`

Takeaway de la Sección 2

La sintaxis a nivel de línea de Markdown optimiza la legibilidad visual, pero proporciona una vía de escape segura hacia HTML puro cuando se requiere control absoluto sobre los atributos del elemento.

Encabezados de Bloque y Reglas de Espaciado Estricto

✓ Do this

Try to put a blank line before...

```
# Heading level 1
```

...and after a heading.

✗ Don't do this

Without blank lines, this might not look right.

```
#Heading
```

Don't do this!

El símbolo # mapea directamente a las etiquetas <h1> hasta <h6> de HTML. La compatibilidad entre distintos parsers exige estrictamente un espacio después del símbolo y líneas en blanco delimitadoras.

Párrafos, Saltos de Línea y Citas Estructurales

First line of text.
Second line of text.

- > Primer nivel de cita.
 - >> Segundo nivel anidado.
- > * Puede contener listas y ****énfasis****.

First line of text.
Second line of text.

- Primer nivel de cita.
 - Segundo nivel anidado.
 - Puede contener listas y **bold text**.

Los saltos de línea se controlan con doble espacio final o la etiqueta **
**. Las citas (Blockquotes) actúan como contenedores genéricos que soportan elementos Markdown anidados en su interior.

La Arquitectura de Anidamiento en Listas

1. Primer elemento de la lista.

 Párrafo anidado que requiere 4 espacios de sangría.

 → 4 espacios (o 1 tab)

2. Segundo elemento de la lista.

 Código bloque anidado que requiere 8 espacios.

 → 8 espacios (o 2 tabs)

Takeaway de la Sección 3

El espaciado horizontal en Markdown no es cosmético; es la estructura sintáctica que define la jerarquía del árbol de nodos (DOM) en el documento renderizado.

Sintaxis Extendida: Estructuras de Datos Tabulares

```
| Syntax | Description |  
| :--- | :---: |  
| Header | Title |  
| Row 1 | Data |
```

Syntax	Description
Header	Title
Row 1	Data

Las tablas requieren tres o más guiones (---) y barras verticales (|). Para escapar una barra vertical dentro de una celda sin romper la estructura de la tabla, se utiliza el código de entidad HTML `|`.

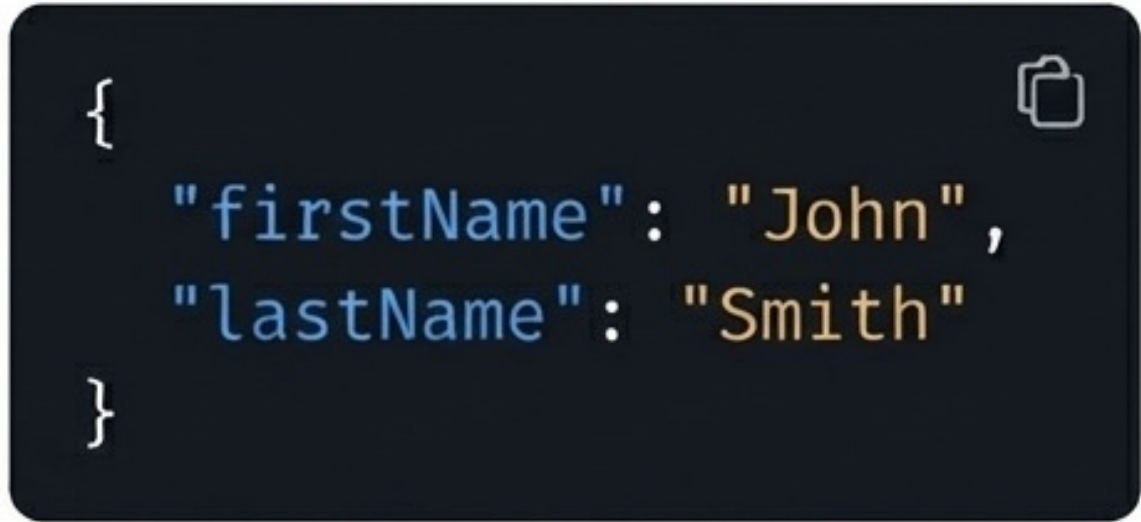
Fenced Code Blocks y Resaltado Sintáctico

```
```json
{ "firstName": "John",
 "lastName": "Smith" }
```
```

~~~

Código sin resaltar

~~~



```
{
  "firstName": "John",
  "lastName": "Smith"
}
```

A diferencia del código con sangría básica, los bloques "vallados" no requieren espacios previos y permiten especificar el lenguaje de programación en la primera línea para activar el resaltado léxico en el parser.

Gestión de Tareas y Anotaciones Extendidas

- `[ ]</span>` Tarea pendiente
- `[x]</span>` Tarea completada

Texto con una nota al pie^{`[^1]</span>`}.

`[^1]</span>`: Definición detallada de la nota.

☐ Tarea pendiente

☒ ~~Tarea completada~~

Texto con una nota al pie¹.

1. Definición detallada de la nota.

Takeaway de la Sección 4

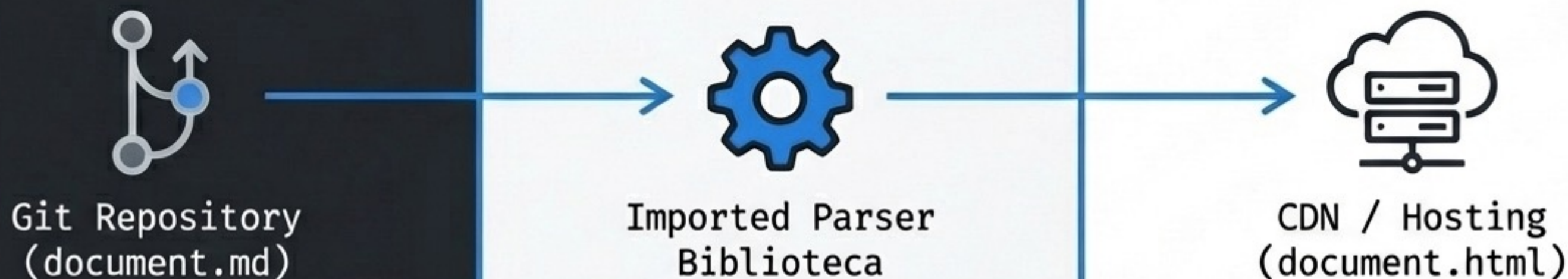
La sintaxis extendida (como GitHub Flavored Markdown) transforma una herramienta básica de formateo en un entorno robusto para la ingeniería ágil, la revisión de código y la documentación técnica profunda.

El Ecosistema de Herramientas: Matriz de Artefactos

| Modern IDE / Split-Pane Editor | |
|--------------------------------|------------------------------|
| Artefacto (Salida) | Herramientas Recomendadas |
| Sitios Web Estáticos | Jekyll, Blot, Ghost |
| Documentación de Software | MkDocs, Docusaurus, VuePress |
| Libros Electrónicos (PDF/EPUB) | Leanpub, Pandoc |
| Presentaciones (Slides) | Marp, Remark, Deckset |

Markdown alimenta una industria masiva de Generadores de Sitios Estáticos (SSG) y plataformas de publicación impulsadas por control de versiones.

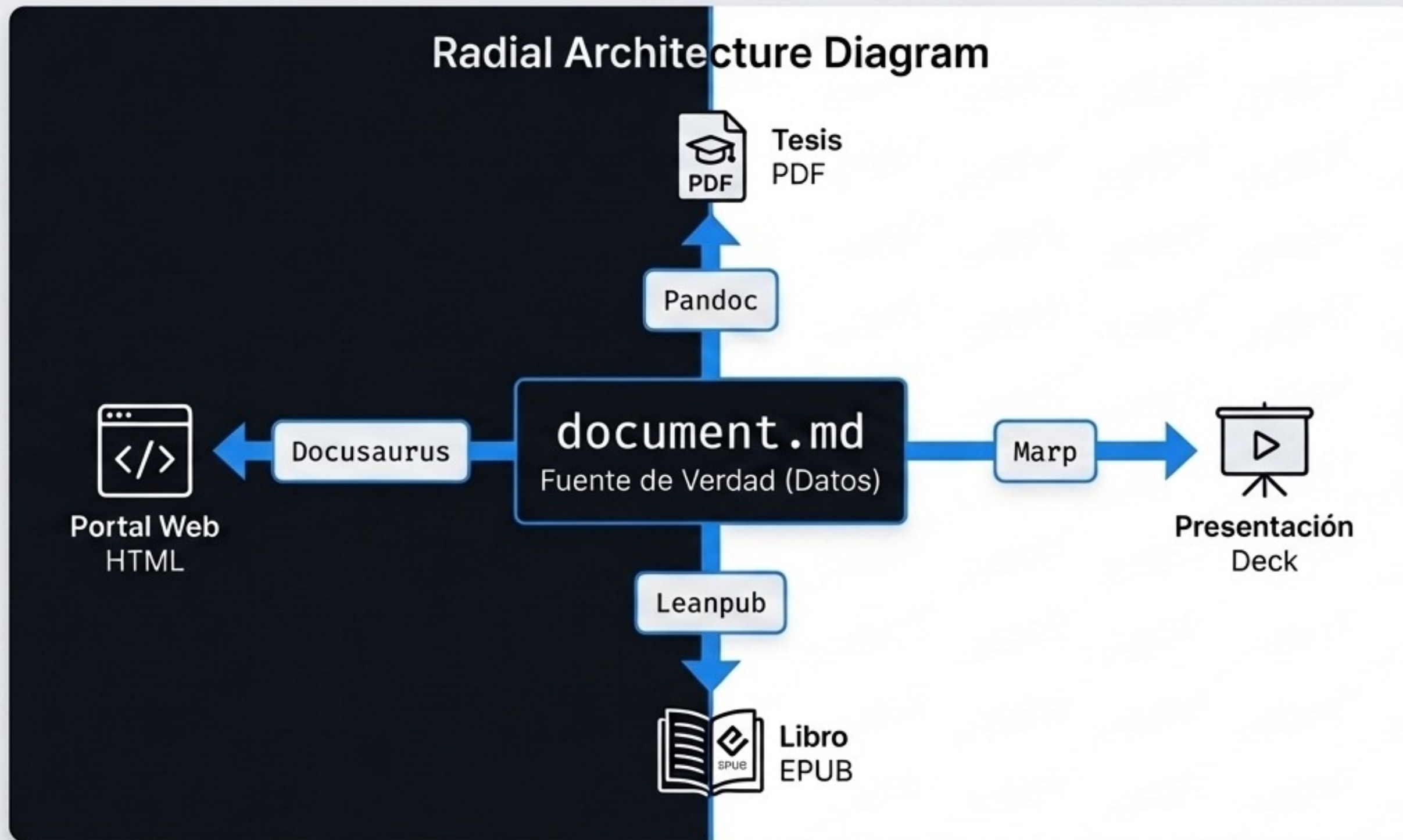
Integración de Bibliotecas y Parsers



Takeaway de la Sección 5

Markdown no es solo para usuarios finales. Como ingeniero de software, integrarás bibliotecas para parsing directamente en tus aplicaciones para permitir a los usuarios gestionar contenido estructurado de forma segura.

Síntesis: La Capa de Abstracción Universal



Markdown es la máxima expresión del principio arquitectónico de Separation of Concerns. Escribes la lógica de los datos una sola vez; el ecosistema lo compila en cualquier formato de presentación necesario.

Referencias Oficiales

Modern IDE / Split-Pane Editor

- **"Basic Syntax | Markdown Guide"**. Creado por Matt Cone (2026).
<https://www.markdownguide.org/basic-syntax>
- **"Extended Syntax | Markdown Guide"**.
<https://www.markdownguide.org/extended-syntax>
- **"Getting Started | Markdown Guide"**.
(Exploración del origen por John Gruber en 2004). <https://www.markdownguide.org/getting-started>

Las herramientas referenciadas (Jekyll, Docusaurus, MkDocs, Marp, Leanpub, Blot) representan proyectos de código abierto del ecosistema actual de desarrollo de software.

</html>

> Markdown: Escribe una vez,
compila a cualquier formato.

Markdown: Escribe
una vez, compila a
cualquier formato.

Gitflow Workflow: Arquitectura de Ramas y Control de Versiones

Una guía profunda para la ingeniería de software



Agenda del Recorrido

-
- 1. Introducción al Modelo Gitflow 
- 2. Las Ramas Principales (main y develop)  
- 3. Ramas de Funcionalidad (feature) 
- 4. Ramas de Lanzamiento (release) 
- 5. Ramas de Corrección Rápida (hotfix) 
- 6. Síntesis y Matriz de Comparación 
- 7. Resumen General 



El Contexto: Trunk-based vs. Gitflow



Desarrollo Basado en el Tronco (Trunk-based)



El estándar moderno ágil. Una sola línea continua con integraciones pequeñas y rápidas.

Modelo Gitflow (Vincent Driessen)



Modelo estricto con múltiples ramas de larga duración y roles específicos.

Contexto de la Industria: Aunque es un modelo heredado, Gitflow sigue siendo fundamental para proyectos con ciclos de lanzamiento programados (scheduled release cycles) y entregas continuas.



La Fundación: El Sistema de Rama Dual



Rama main

Almacena exclusivamente el historial de lanzamientos oficiales en producción. Protegida y estable.

Rama develop

Sirve como la rama de integración. Contiene el historial completo del proyecto y es el punto base para cualquier nueva funcionalidad.



Implementando la Fundación

Anotación: La extensión (biblioteca) automatiza la creación del entorno con un solo comando.

Git Estándar

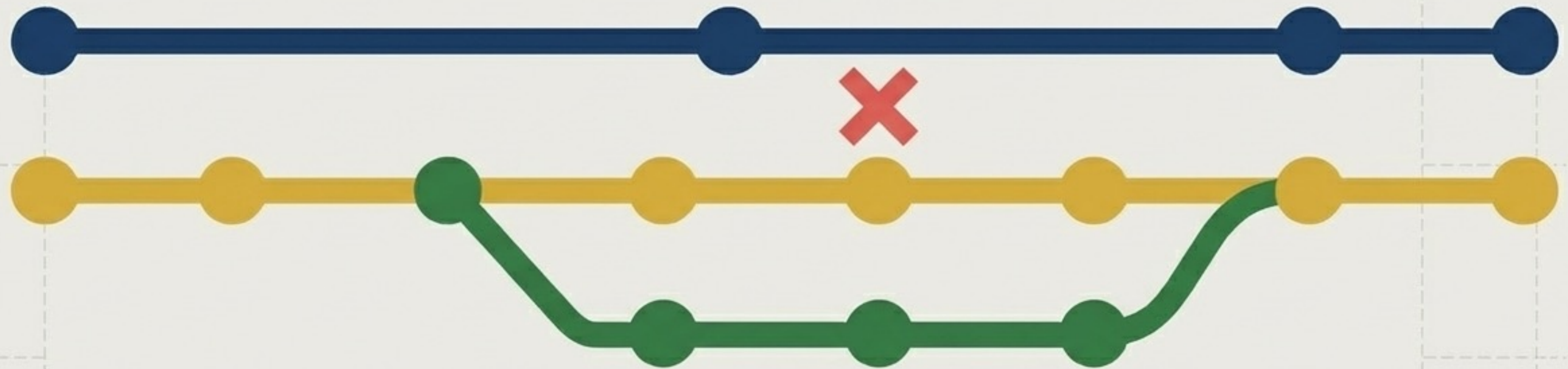
```
git branch develop  
git push -u origin develop
```

Biblioteca git-flow

```
git flow init
```

Conclusión (Takeaway): develop es el nuevo punto de partida para todo el código nuevo, protegiendo a main de código no probado.

Aislamiento de Trabajo: Ramas de Funcionalidad



Aislamiento del Desarrollo

Cada nueva característica reside en su propia rama aislada, permitiendo trabajo en paralelo sin colisiones.

Regla de Oro

Las ramas feature utilizan develop como padre y se fusionan de nuevo en develop. Jamás interactúan directamente con main.

Ciclo de Vida de una Funcionalidad

Creación de Rama

```
# Crear la rama
git checkout -b feature_branch develop
# O usando la biblioteca:
git flow feature start feature_branch
```

Finalización de Rama

```
# Finalizar la rama
git checkout develop
git merge feature_branch
# O usando la biblioteca:
git flow feature finish feature_branch
```



Nota de la Industria:

En equipos modernos, el paso de 'merge' casi siempre se gestiona mediante un Pull Request (PR) para garantizar la revisión de código por pares.

Conclusión (Takeaway): Las ramas feature aíslan el desarrollo y siempre nacen y mueren en **develop**.

Congelamiento de Código: Ramas de Lanzamiento



Preparación para Producción

Se crea cuando develop tiene suficientes características terminadas para el próximo lanzamiento programado.

Restricción Estricta

A partir de este punto, no se permiten nuevas funcionalidades en esta rama—solo corrección de errores (bug fixes) y preparación de documentación.

La Fusión Dual: Asegurando el Código

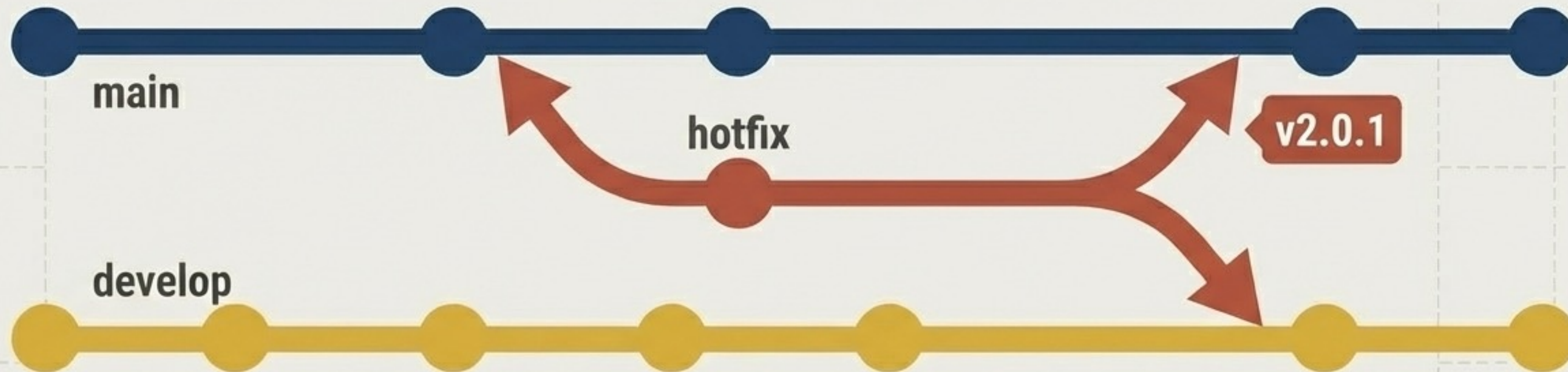
```
git checkout main
git merge release/0.1.0

git checkout develop
git merge release/0.1.0
```



Conclusión (Takeaway): Las ramas release permiten pulir la versión actual mientras otro equipo comienza simultáneamente las funcionalidades de la siguiente versión.

La Excepción: Ramas de Corrección Rápida



Parches de Emergencia

Diseñadas para resolver problemas críticos en producción de manera inmediata, saltándose la cola de espera.

La Excepción a la Regla

Es la única rama que se bifurca directamente de main, actuando como una rama de lanzamiento ad hoc.

Ejecutando el Parche de Emergencia



```
# Crear el parche directo desde producción
git checkout -b hotfix_branch main
# Biblioteca: git flow hotfix start hotfix_branch

[... se repara el error crítico ...]

# Fusión en main y develop
# Biblioteca: git flow hotfix finish hotfix_branch
```



¡Precaución! Alterando el historial de producción directamente.

Conclusión (Takeaway): Las ramas hotfix son el único canal directo para alterar producción de emergencia sin interrumpir el flujo de desarrollo normal.

Matriz de Reglas de Ramificación

| Tipo de Rama | Se origina de | Se fusiona en | Propósito |
|--------------|---------------|----------------|--------------------------------------|
| feature | develop | develop | Nuevas características aisladas |
| release | develop | main & develop | Preparación y pulido para producción |
| hotfix | main | main & develop | Parches críticos de emergencia |

Esta matriz es el motor lógico central del flujo de trabajo.

Arquitectura Completa en Ejecución



Síntesis: El ciclo de vida completo de Gitflow. Observa cómo el sistema garantiza el desarrollo paralelo, absorbe emergencias en producción y estructura entregas programadas sin colisiones de código.

Resumen General y Mejores Prácticas

Ventajas (Pros)

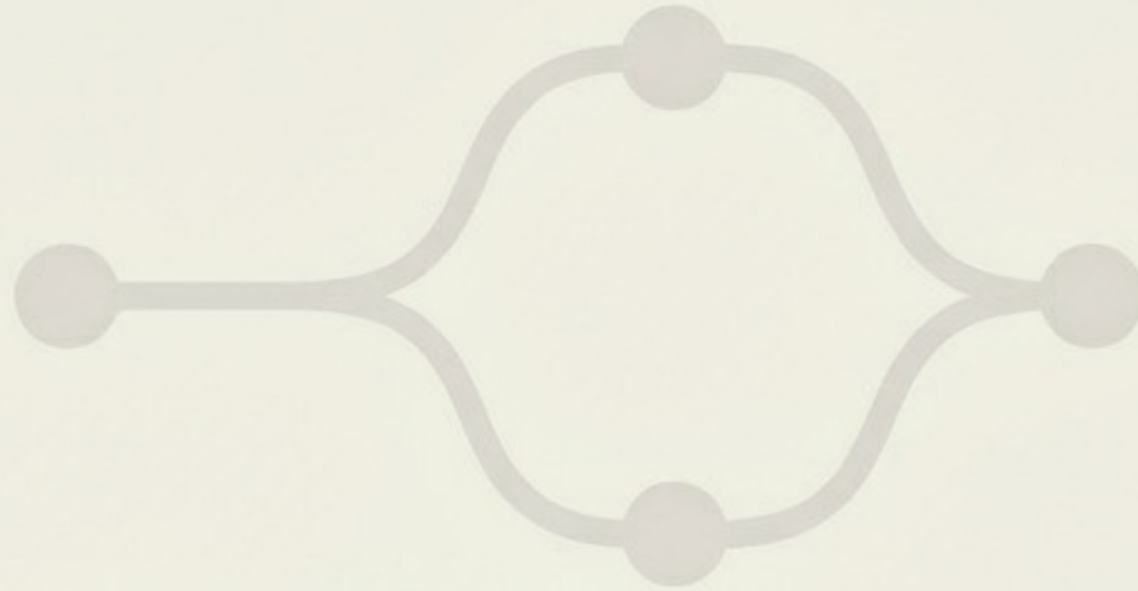
- ✓ Excelente control sobre versiones a producción.
- ✓ Canales dedicados e independientes para emergencias (hotfixes).
- ✓ Fases de desarrollo claramente definidas.

Desafíos (Cons)

- ⚠ Complejidad excesiva para Integración Continua (CI/CD) moderna.
- ⚠ Ramas de larga duración pueden generar conflictos de fusión severos.



Recordatorio: Gitflow asigna roles estrictos para proyectos con lanzamientos programados. Utiliza la biblioteca (extensión) `git-flow` para automatizar estos pasos y evitar errores humanos en la terminal.



Referencias:

Gitflow Workflow | Atlassian Git Tutorial, Atlassian (2026).
Gitflow Workflow | Atlassian Git Tutorial, Atlassian (2026).
Extraído de la documentación oficial de Git de Atlassian.

Para profundizar en estrategias modernas de ramas o uso de la biblioteca git-flow, consulte la **base de conocimientos oficial de repositorios remotos**.

Fin del Recorrido.



Especificación Oficial / Arquitectura de Software

Versionado Semántico 2.0.0 (SemVer)

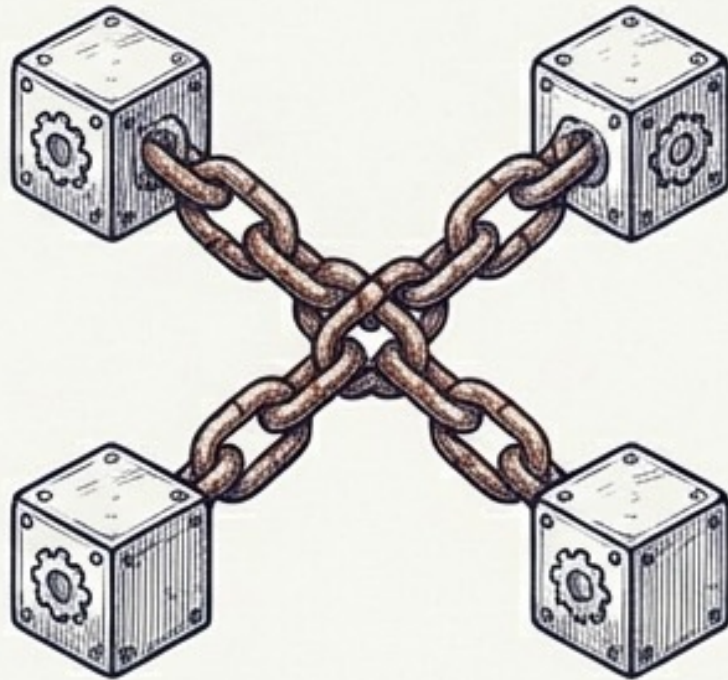
Masterclass: De la Teoría a la Ingeniería Pragmática

Agenda

- El Problema (El Infierno de las Dependencias)
- La Especificación (La Anatomía de una Promesa)
- Casos Reales (El Ciclo de Vida y Reglas Avanzadas)
- Síntesis (Herramientas y Referencias)

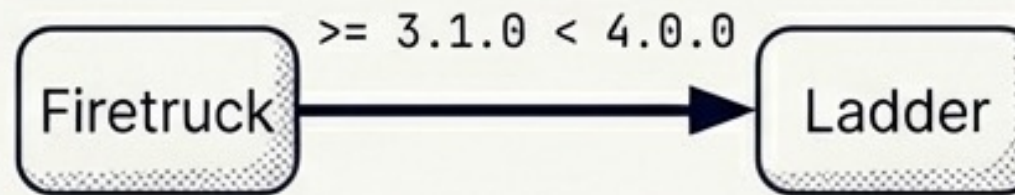
El Infierno de las Dependencias

Bloqueo de Versión



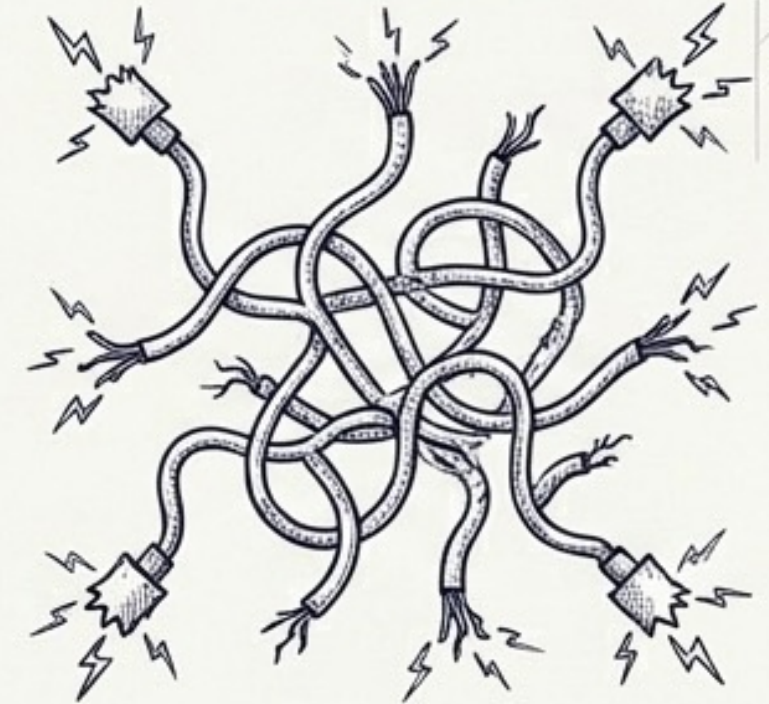
Dependencias demasiado estrictas. Imposibilidad de actualizar sin romper el ecosistema.

La Solución



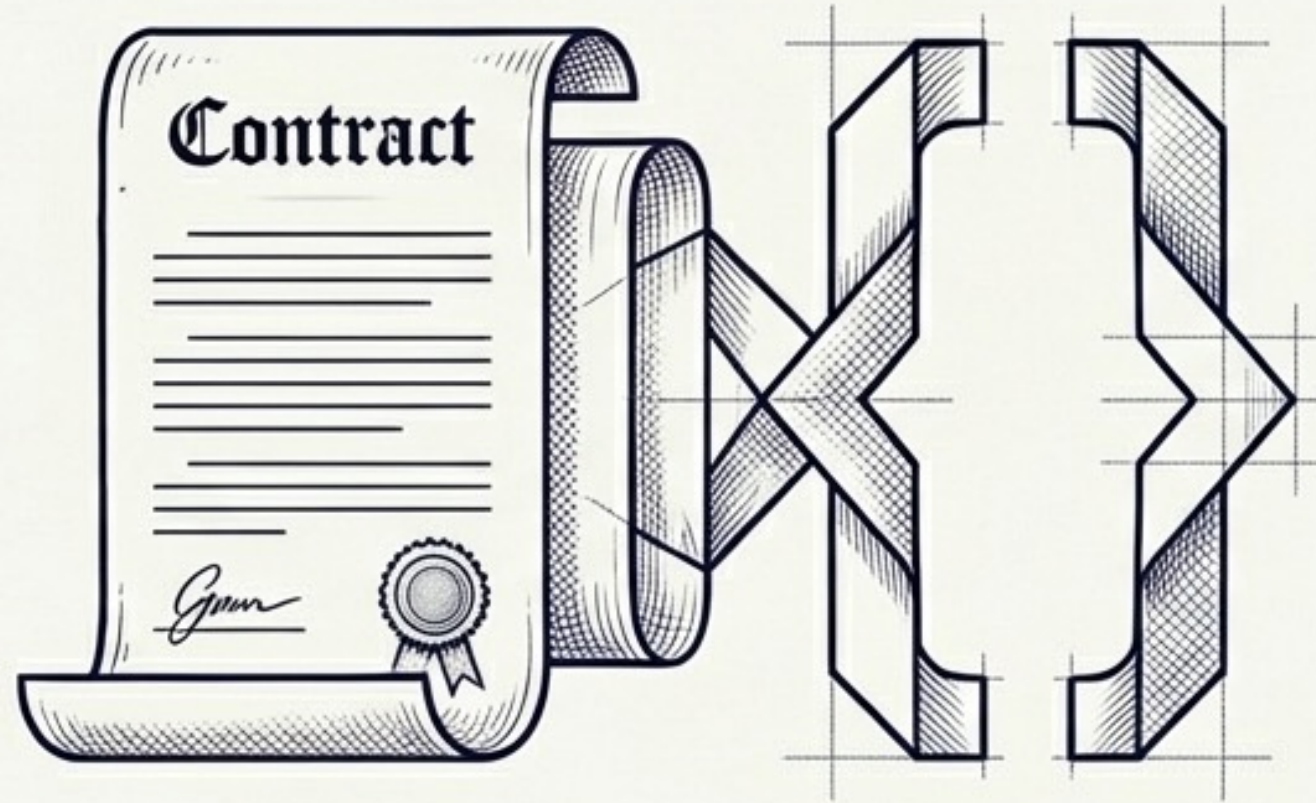
La solución: Especificaciones flexibles pero seguras para la biblioteca.

Promiscuidad de Versión



Dependencias demasiado flexibles. Rupturas sorpresa en producción.

Takeaway / Sección 1



SemVer no es solo sobre números; es un contrato de comunicación estructurado que depende fundamentalmente de una **API Pública** clara y precisa.

La Anatomía de SemVer (Reglas 2, 6, 7 y 8)

X, Y, Z deben ser enteros no negativos y
NO DEBEN contener ceros a la izquierda.

MAJOR . **MINOR** . **PATCH**

X . **Y** . **Z**

MAJOR (X):

Se incrementa por cambios incompatibles en la API pública.

MINOR (Y):

Se incrementa por nueva funcionalidad retrocompatible o deprecaciones.

PATCH (Z):

Se incrementa por correcciones de errores (bug fixes) retrocompatibles.

La Matriz de Incremento

| | MAJOR | MINOR | PATCH |
|-----------------------------------|--------------------------------------|--------------------------------------|---|
| Gatillo: | Cambios incompatibles en la API. | Nueva funcionalidad retrocompatible. | Solo corrección de errores (bug fixes). |
| Impacto en Código Privado: | Puede incluir cambios minor/patch. | Mejoras sustanciales permitidas. | Cambios puramente internos. |
| Cascada de Reseteo: | MINOR y PATCH DEBEN reiniciarse a 0. | PATCH DEBE reiniciarse a 0. | Ninguno. |

Metadatos y Extensiones (Reglas 9 y 10)

1.0.0

-rc.1

+20130313144700

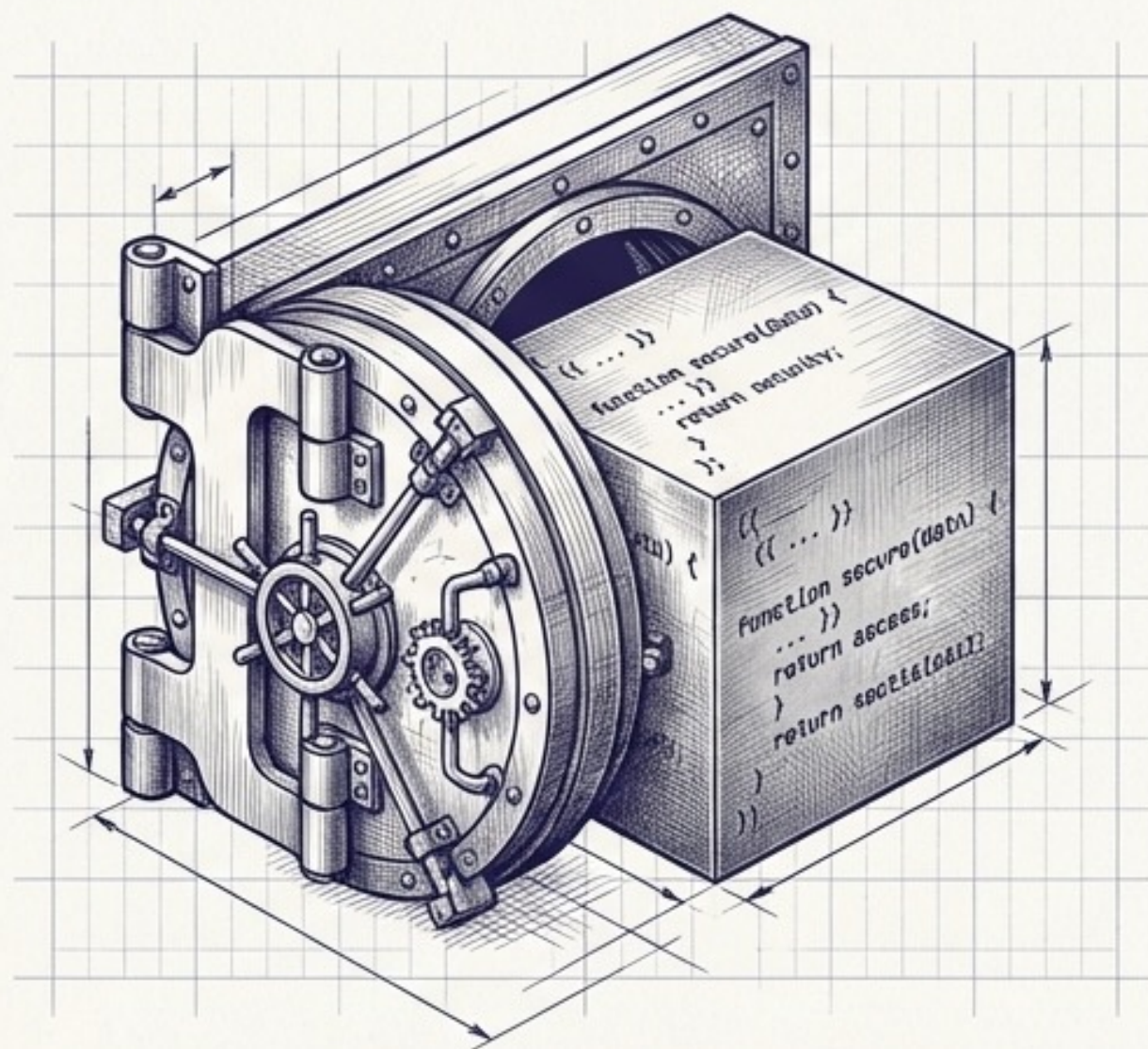
Versión Pre-lanzamiento.

Precedida por un guion.
Indica inestabilidad.
Menor precedencia que la
versión normal.
(Ej: 1.0.0-alpha.1)

Metadatos de Construcción.

Precedida por un signo más.
DEBE ser ignorado al determinar
la precedencia.
Dos versiones que solo difieren
aquí son funcionalmente
idénticas en orden.

Takeaway / Sección 2



Inmutabilidad Absoluta (Regla 3). Una vez que se libera una versión empaquetada, su contenido NO DEBE ser modificado jamás. Cualquier modificación exige el lanzamiento de una nueva versión.

El Embudo de Precedencia

(Regla 11)

Comparación Numérica
(Major, Minor, Patch)

Major

Minor

Patch

Pre-lanzamiento vs Normal
(1.0.0-alpha < 1.0.0)

1.0.0-alpha

1.0.0

Orden Lexicográfico vs
Numérico (ASCII)

ASCII

Metadatos de
compilación
(Ignorados)

+20130313144700

1.0.0-alpha

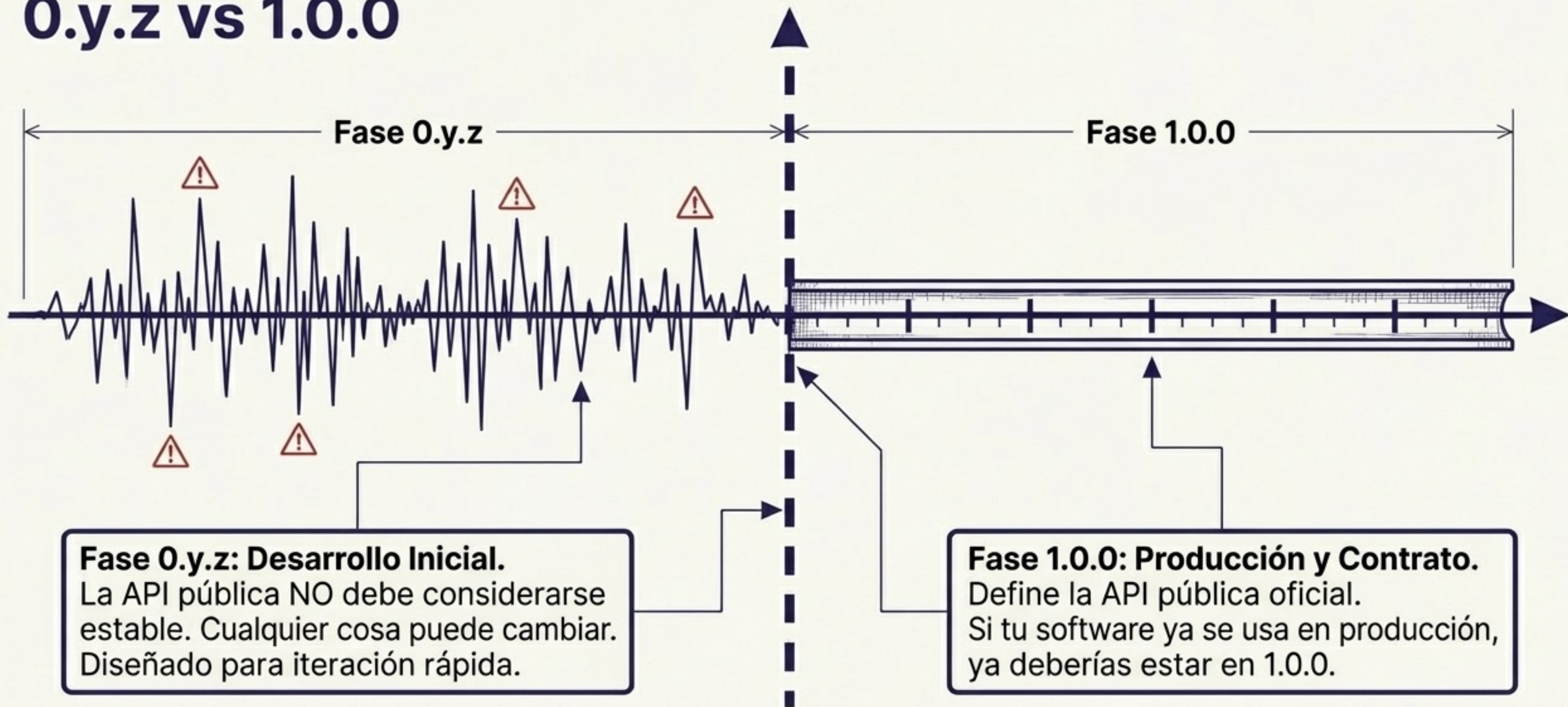
1.0.0-alpha.1

1.0.0-beta

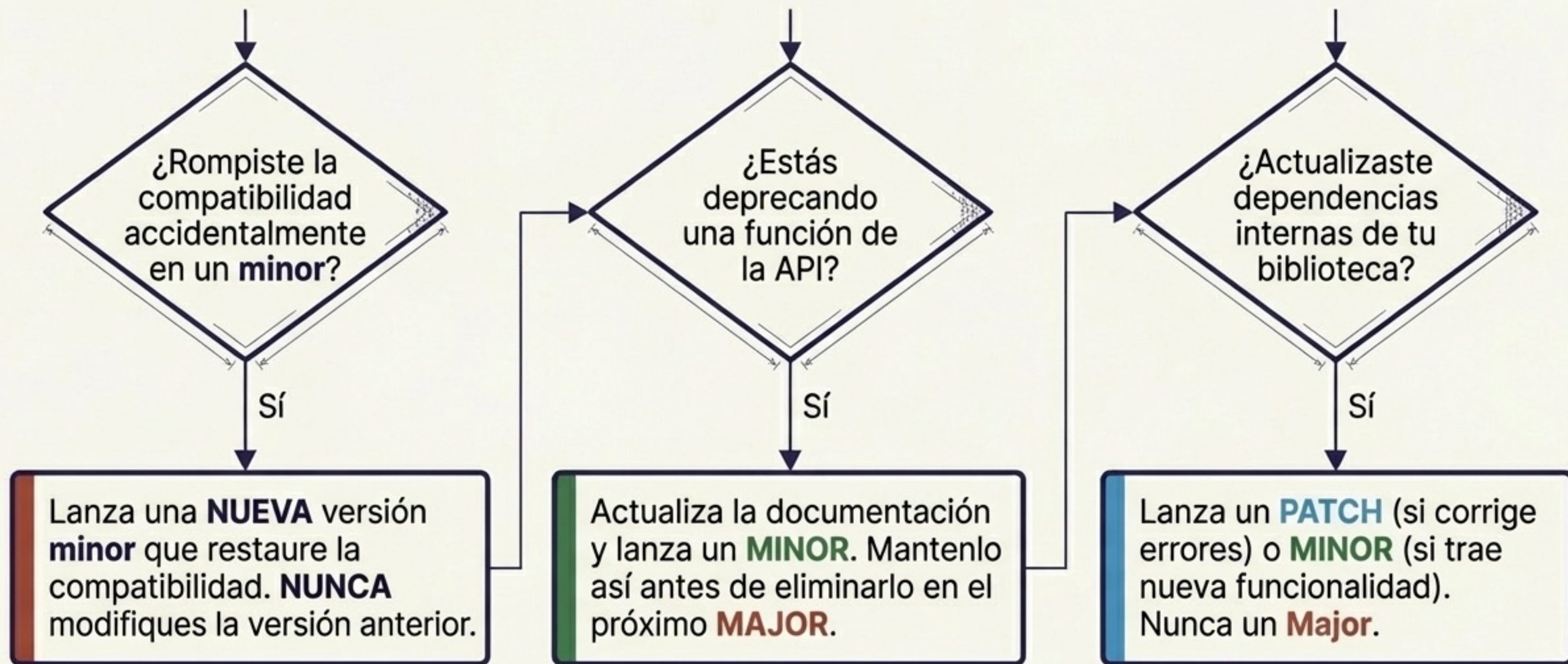
1.0.0-rc.1

1.0.0

El Umbral de la Estabilidad: 0.y.z vs 1.0.0



Decisiones Pragmáticas (Síntesis del FAQ)



Implementación y Herramientas

El Prefijo "v"

¿Es v1.2.3 una versión semántica? **No.**

El formato **oficial** es estrictamente numérico: **1.2.3.**

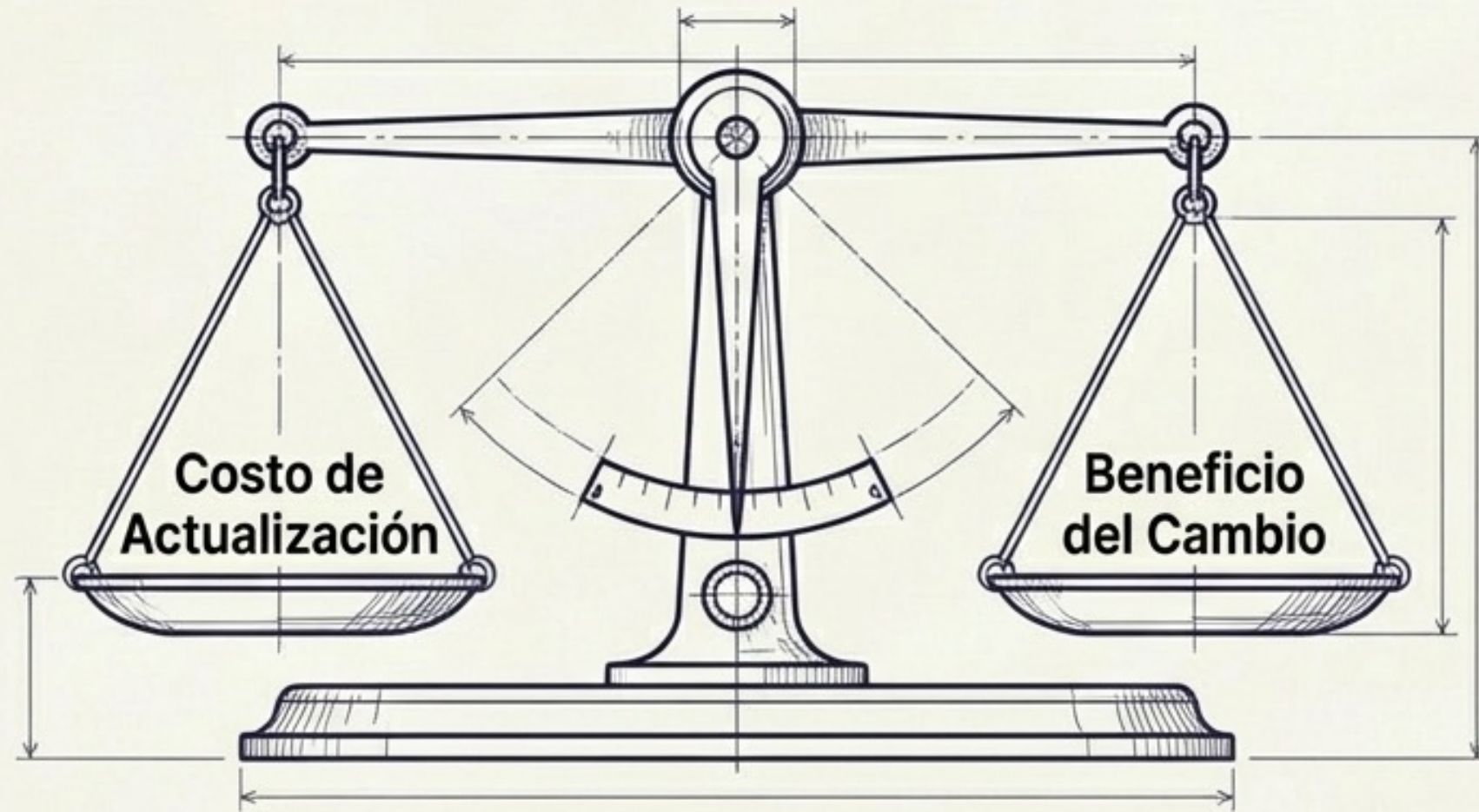
Usar v1.2.3 es un estándar común para **Git tags**, pero la versión SemVer subyacente sigue siendo **1.2.3.**

Validación Oficial (RegEx)

```
^(?P<major>0|[1-9]\d*)\. (?P<minor>0|[1-9]\d*)\.(?P<patch>0|[1-9]\d*)
(?:- (?P<prerelease>(?:0|[1-9]\d*|
\d*[a-zA-Z-][0-9a-zA-Z-]*))
(?:\.(?:0|[1-9]\d*|\d*[a-zA-Z-][0-9a-zA-Z-]*))*))?)
(?:\.(?:\d*[a-zA-Z-][0-9a-zA-Z-]*+))?)?$
```

RegEx Oficial con grupos de **captura con nombre** (Compatible con PCRE, Python, Go).

Takeaway / Sección 3



La Responsabilidad del Major.

Forzar un incremento Major por cambios mínimos incompatibles requiere **previsión**. Obliga al desarrollador a evaluar responsablemente la relación costo/beneficio para los usuarios de su biblioteca.

SemVer 2.0.0: Resumen Ejecutivo

MAJOR

Rompe la API pública.

MINOR

Añade funciones (Retrocompatible).

PATCH

Corrige bugs (Retrocompatible).

Autor Original: Tom Preston-Werner (Creador de Gravatar, Co-fundador de GitHub).

Licencia:
Creative Commons — CC BY 3.0.



Para leer la especificación completa o usar las pruebas RegEx, visita: **semver.org**



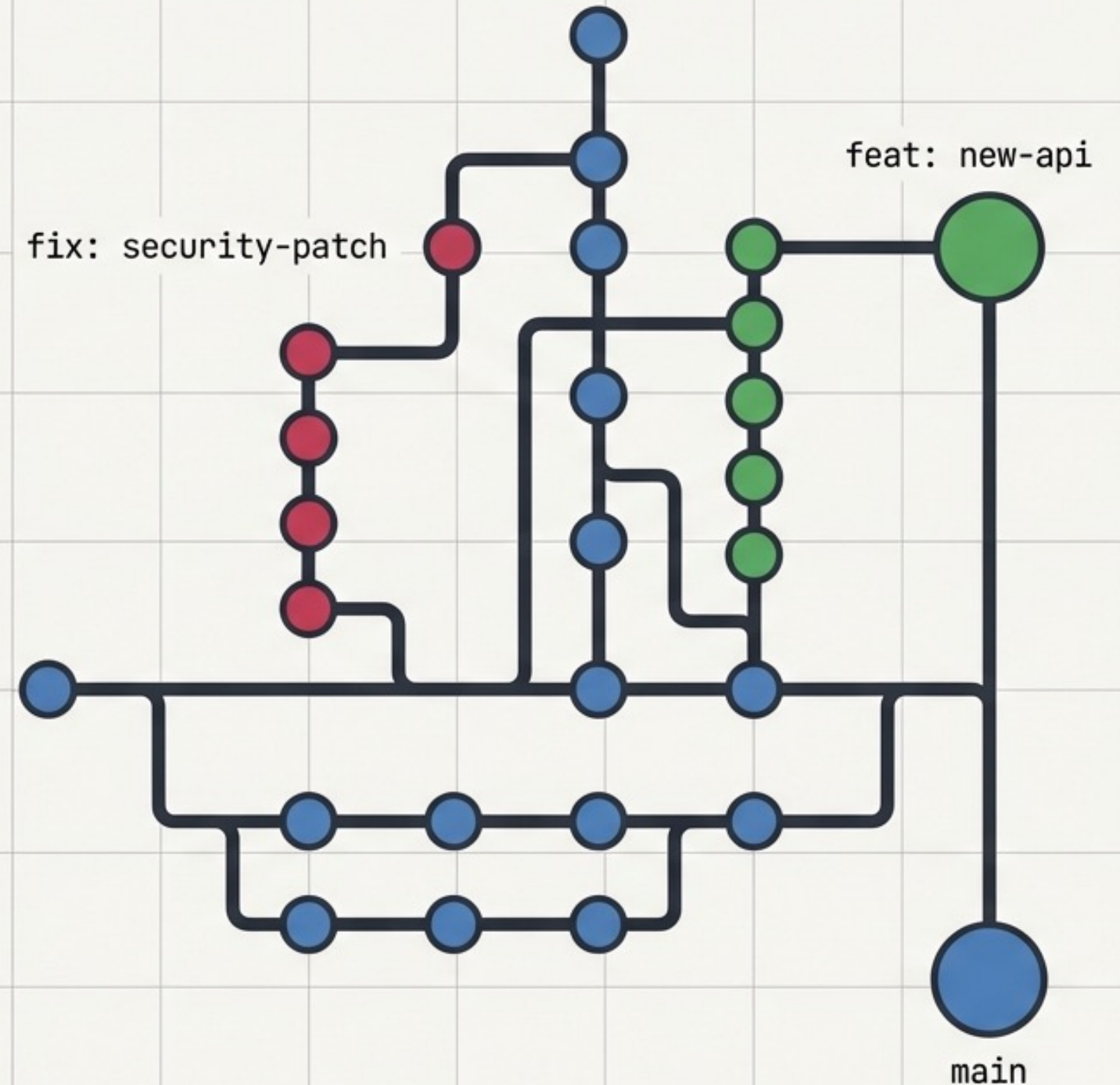
**Comunica tus intenciones.
Evita el caos.**

Versionado Semántico 2.0.0.

Conventional Commits 1.0.0: Código como Comunicación

Estandarizando el historial de Git para la automatización y el trabajo en equipo en la ingeniería de software.

Especificación Oficial v1.0.0



Índice del Proyecto



01_Anatomía_y_Sintaxis
Deconstruyendo el mensaje



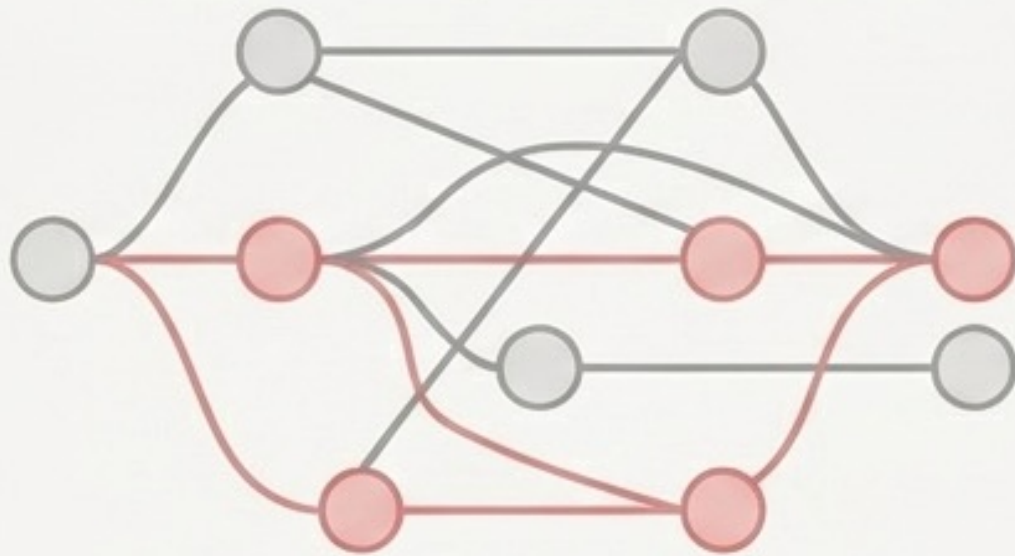
02_La_Especificación_RFC2119
Reglas estrictas de implementación



03_SemVer_y_Automatización
Impacto en integración continua

El Paradigma: De Notas de Texto a Payloads de API

El Caos / Historial Tradicional



```
fixed the bug  
wip  
updated stuff  
oops typo
```

Imposible de analizar por máquinas, contexto perdido para humanos.

El Orden / Conventional Commits



```
fix(auth): resolve token expiration  
feat(api): add user endpoint  
docs: update readme
```

Intención explícita, historial estructurado, automatización habilitada para tu biblioteca o aplicación.

Anatomía del Commit Message: Desglose Estructural

type(scope)!: description \n\n body \n\n footer

type
(Required):

Define la naturaleza del cambio (e.g., feat, fix).

(scope)
(Optional):

Sustantivo entre paréntesis que indica la sección del código afectada (e.g., parser).

!
(Optional):

Indicador visual inmediato de un cambio incompatible (Breaking Change).

description
(Required):

Resumen corto tras dos puntos y un espacio.

body
(Optional):

Párrafos de forma libre separados por líneas en blanco para contexto adicional.

footer
(Optional):

Metadatos tipo git trailer (e.g., Reviewed-by: Z).

“ Un commit convencional estructura la intención del desarrollador tanto para humanos como para máquinas.”

La Sección 01 demuestra que escribir código es solo la mitad del trabajo; comunicarlo correctamente es lo que permite el trabajo en equipo a gran escala.

Vocabulario Estándar: Tipos Principales

feat

Introduce una nueva característica a la base de código.

fix

Corrige un error (bug) en tu biblioteca o aplicación.

Tipos Adicionales Recomendados - Convención Angular

build: Cambios en el sistema de construcción o dependencias externas.

chore: Mantenimiento general, sin cambios en el código de producción.

ci: Cambios en archivos de configuración y scripts de CI.

docs: Actualizaciones exclusivas de documentación.

style: Cambios que no afectan el significado del código (espacios, formato).

refactor: Cambio de código que ni corrige un error ni añade una característica.

perf: Mejora de rendimiento.

test: Adición o corrección de pruebas.

Rompiendo la API: BREAKING CHANGES

Los cambios incompatibles deben señalarse explícitamente. Pueden formar parte de cualquier tipo de commit (feat, fix, chore, etc.).

Método 1: Indicador en el Prefijo

```
feat(api)!: remove deprecated endpoints
```



Un ! justo antes de los dos puntos.
El resumen debe describir la ruptura.

Método 2: Indicador en el Footer

```
BREAKING CHANGE: environment variables  
now take precedence over config files
```



Texto en mayúsculas BREAKING CHANGE en el footer, seguido de : o # y la descripción. Si se usa el !, este footer es opcional.

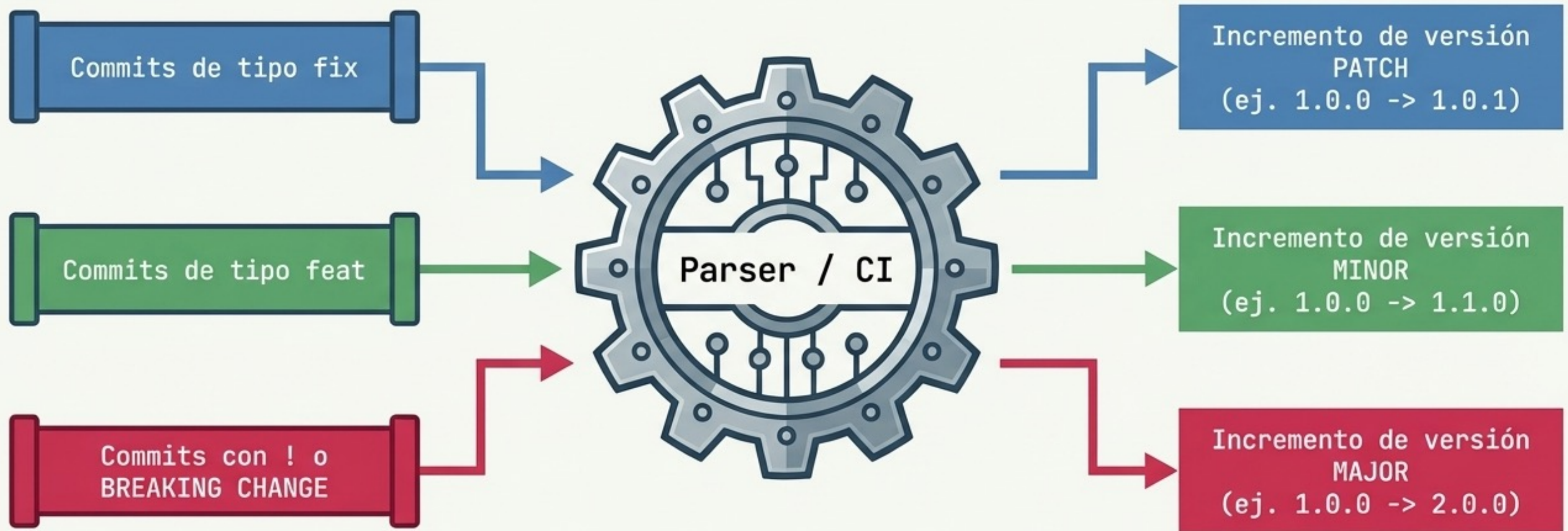
Reglas de Implementación (RFC 2119)

| DO / MUST | DON'T / MUST NOT |
|--|--|
|  Separar el body y el footer con una línea en blanco de la descripción. |  No olvidar el espacio obligatorio después de los dos puntos en el prefijo (ej. <code>fix:error</code> es inválido; debe ser <code>fix: error</code>). |
|  Usar guiones (-) en lugar de espacios para los tokens del footer (ej. <code>Reviewed-by:</code>). Excepción: <code>BREAKING CHANGE</code> . |  No escribir <code>BREAKING CHANGE</code> en minúsculas en el footer. |
|  Tratar las unidades de información como insensibles a mayúsculas/minúsculas. |  No limitar a los desarrolladores: la especificación es flexible, se pueden crear nuevos tipos si el equipo lo necesita. |

La consistencia es vital: usar la sintaxis exacta garantiza que las herramientas de análisis de dependencias de tu biblioteca funcionen sin intervención humana.

La Sección 02 estandariza el formato para preparar el código para la etapa final: la automatización.

El Motor de SemVer: Automatización de Versiones



Esta relación automatiza el salto de versiones basándose estrictamente en los tipos de commits fusionados en la rama principal.

Casos Especiales y Flujos de Trabajo

> ¿Qué pasa si me equivoco de tipo de commit (ej. fix en lugar de feat)?

Respuesta: Antes de hacer merge, usa `git rebase -i` para editar el historial. Una vez liberado, depende de tus herramientas. ¡No es el fin del mundo!

> ¿Deben todos los colaboradores usar esta convención?

Respuesta: ¡No! Los mantenedores principales pueden usar la estrategia de Squash and Merge en los Pull Requests para limpiar el historial sin cargar de trabajo a los colaboradores casuales.

> ¿Cómo se manejan los commits de reversión (revert)?

Respuesta: Se recomienda usar el tipo revert y un footer que referencie el SHA del commit que se está revirtiendo.

Conventional Commits es el puente indispensable entre el código escrito y el despliegue automatizado continuo.

La Sección 03 demuestra que, al adoptar esta norma, estás programando tu infraestructura de la misma manera que programas tu aplicación.

Resumen General (Hoja de Referencia)

| Sintaxis | Traducción SemVer | Reglas Clave |
|--|---|--|
| <pre>type(scope)!: description</pre> <pre>\n\n body</pre> <pre>\n\n footer</pre> | <p>fix = PATCH</p> <p>feat = MINOR</p> <p>! = MAJOR</p> | <ol style="list-style-type: none">1. Espacio después de los dos puntos.2. Línea en blanco antes del body/footer.3. Consistencia ante todo. |

Referencias y Ecosistema

```
import { Spec } from  
'conventionalcommits.org/v1.0.0'
```

```
// Especificación oficial  
completa
```

```
import { SemVer } from  
'semver.org'
```

```
// Especificación de  
Versionado Semántico
```

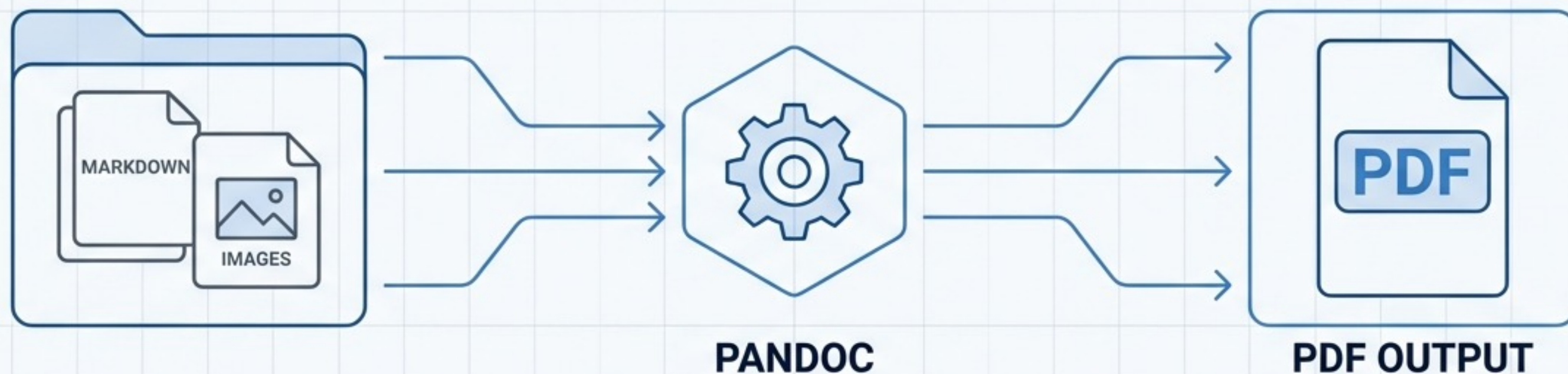
```
import { Tools } from  
'commitlint'
```

```
// Herramientas recomendadas  
para hacer cumplir las  
reglas
```



Estructura tu historial. Automatiza tu futuro.

Fin del documento. Listo para implementación.



Pandoc: De Markdown a PDF a Escala

Guía de Ingeniería para la Compilación de Documentación Multi-Archivo.

Arquitectura del Pipeline Documental

└─ [Sección 1] El Motor Pandoc y Entradas

- └─ Diseño basado en AST
- └─ Inteños
- └─ Orquestación de archivos
- └─ Resolución de recursos
- └─ Configuración declarativa

└─ [Sección 2] Generación Avanzada de PDF

- | └─ Selección del motor
- | └─ Variables phf
- | └─ Variables tipográficas
- | └─ Resaltado de código
- | └─ El Build Script definitivo
- └─ [Resumen y Referencias]

Compilación, no solo Conversión

Pandoc no realiza traducciones directas usando expresiones regulares. Es una biblioteca de Haskell con un diseño estrictamente modular.

Fase 1 (Readers)

Parsea el formato de entrada y construye una representación nativa en memoria.



Fase 2 (Filters)

El AST intermedio es manipulado programáticamente (Lua/JSON).

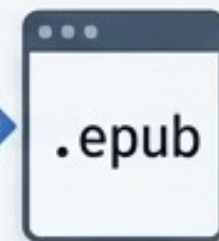


Programmatic Manipulation
(Lua/JSON)

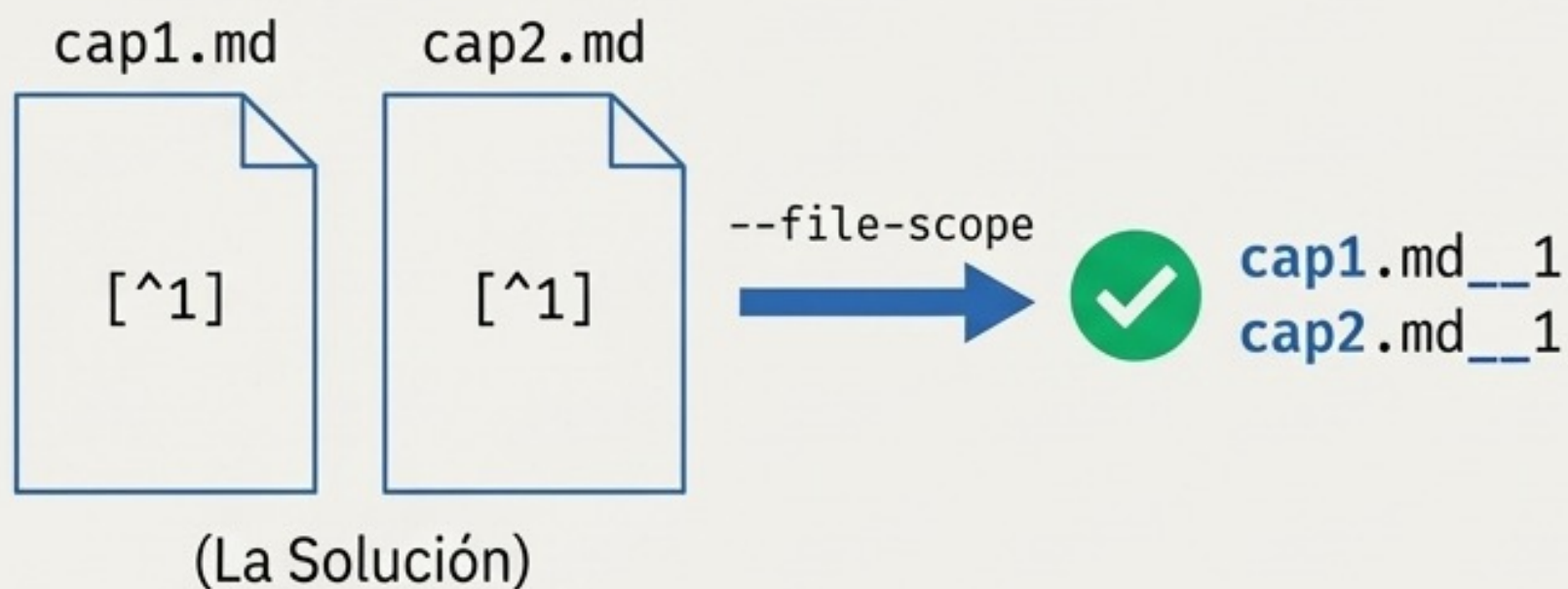
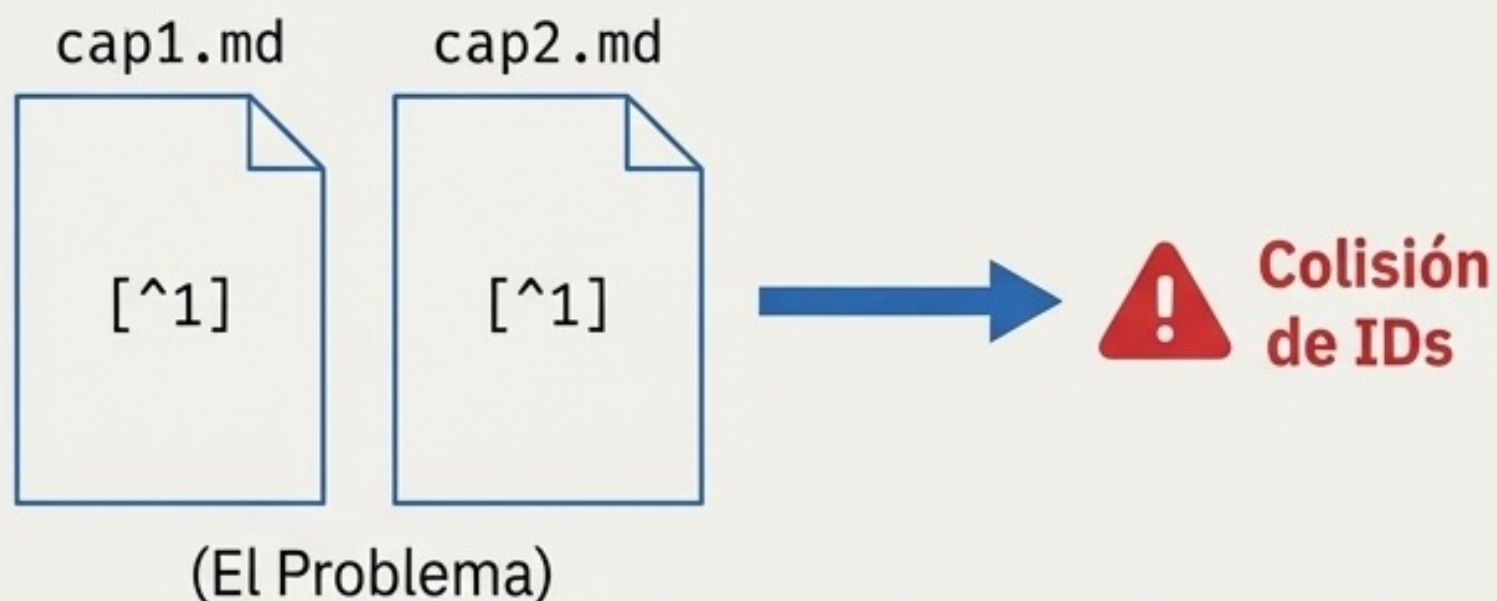
**Pandoc AST
(Abstract Syntax Tree)**

Fase 3 (Writers)

Convierte el AST al formato destino.



Orquestación de Múltiples Archivos



El Problema: Al compilar un directorio, Pandoc concatena los textos antes de parsearlos. Esto provoca colisiones críticas en notas al pie e identificadores.

La Solución: Obligar al Reader a aislar referencias generando prefijos basados en el nombre del archivo.

Best Practice

```
pandoc src/*.md --file-scope -o  
build/output.pdf
```


Trazabilidad de Recursos Internos

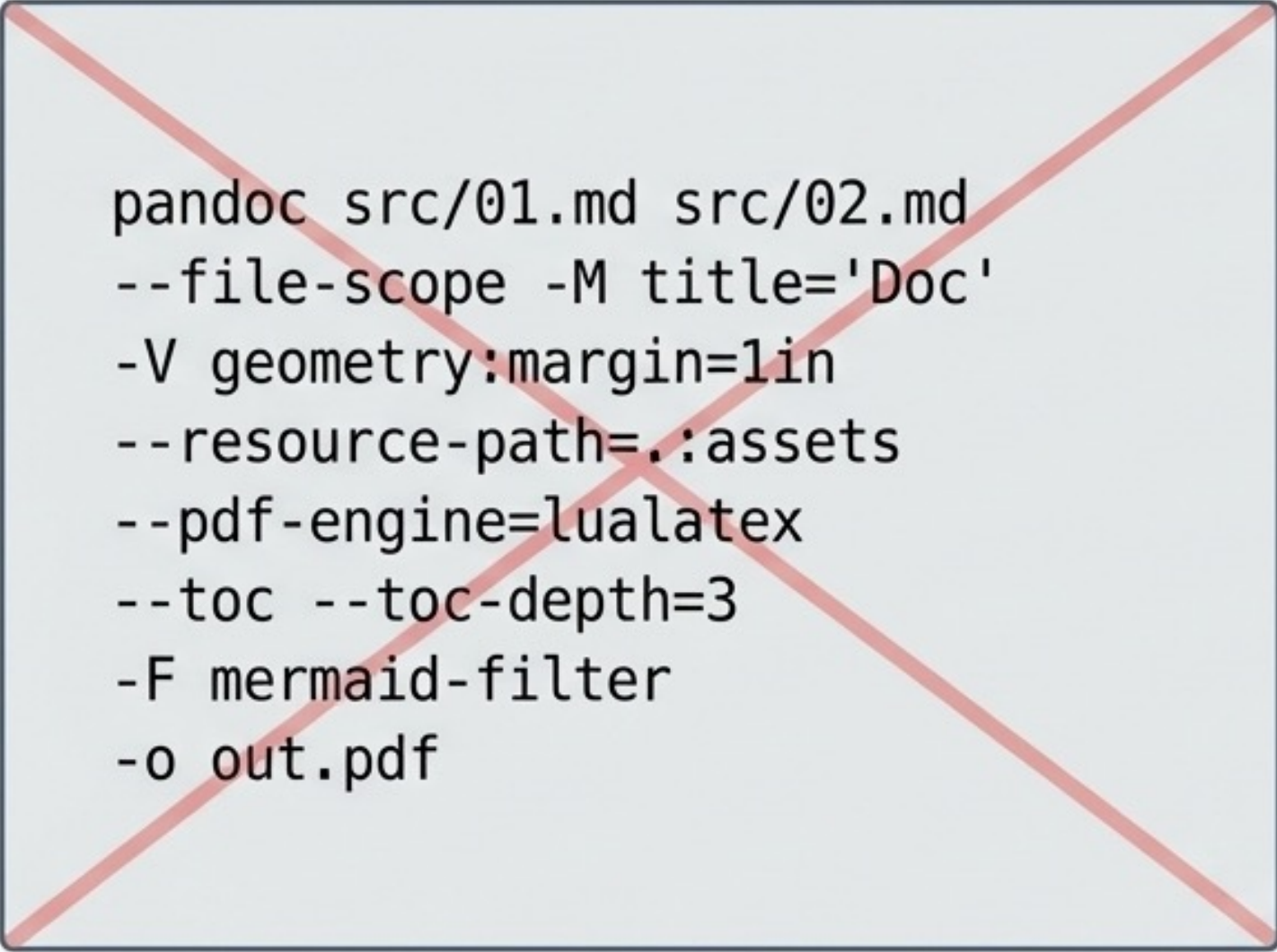
```
/proyecto
├── /src
│   ├── /md
│   │   └── intro.md
│   └── /assets
│       ├── /img
│       │   └── diag.png
```



- **El Desafío:** Los archivos .md en subcarpetas tienen rutas relativas a las imágenes que se rompen al compilar desde el directorio raíz.
- **La Herramienta:** `--resource-path` define una lista de búsqueda (similar al `$PATH` de Linux) para encontrar recursos sin importar la profundidad.

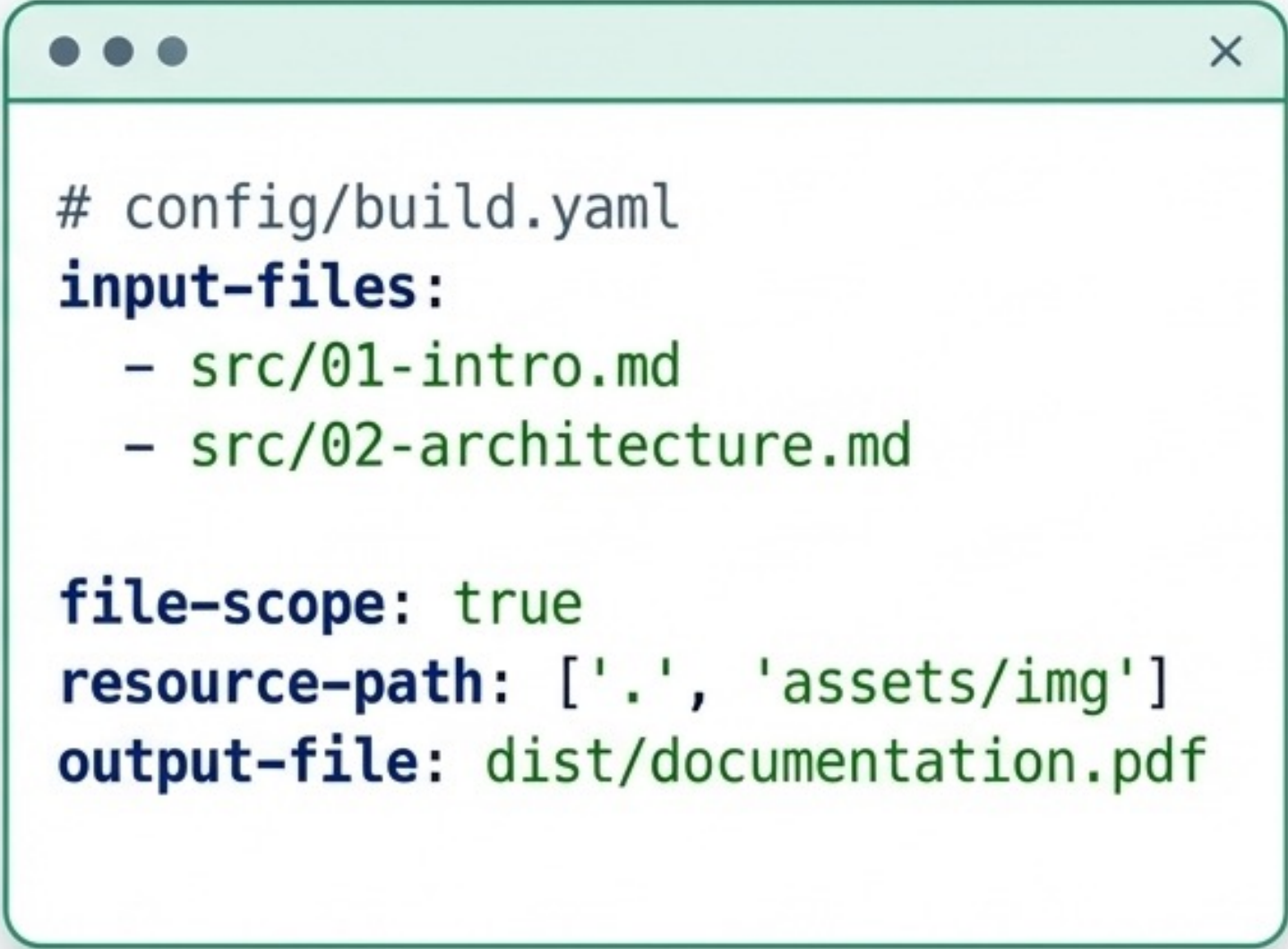
```
# Linux/macOS path separation (:)
pandoc src/**/*.md \
  --resource-path=.:src/assets/img:src/diagrams \
  --extract-media=build/media \
  -o out.pdf
```

Antipatrón: Scripts Frágiles



```
pandoc src/01.md src/02.md
--file-scope -M title='Doc'
-V geometry:margin=1in
--resource-path=./assets
--pdf-engine=lualatex
--toc --toc-depth=3
-F mermaid-filter
-o out.pdf
```

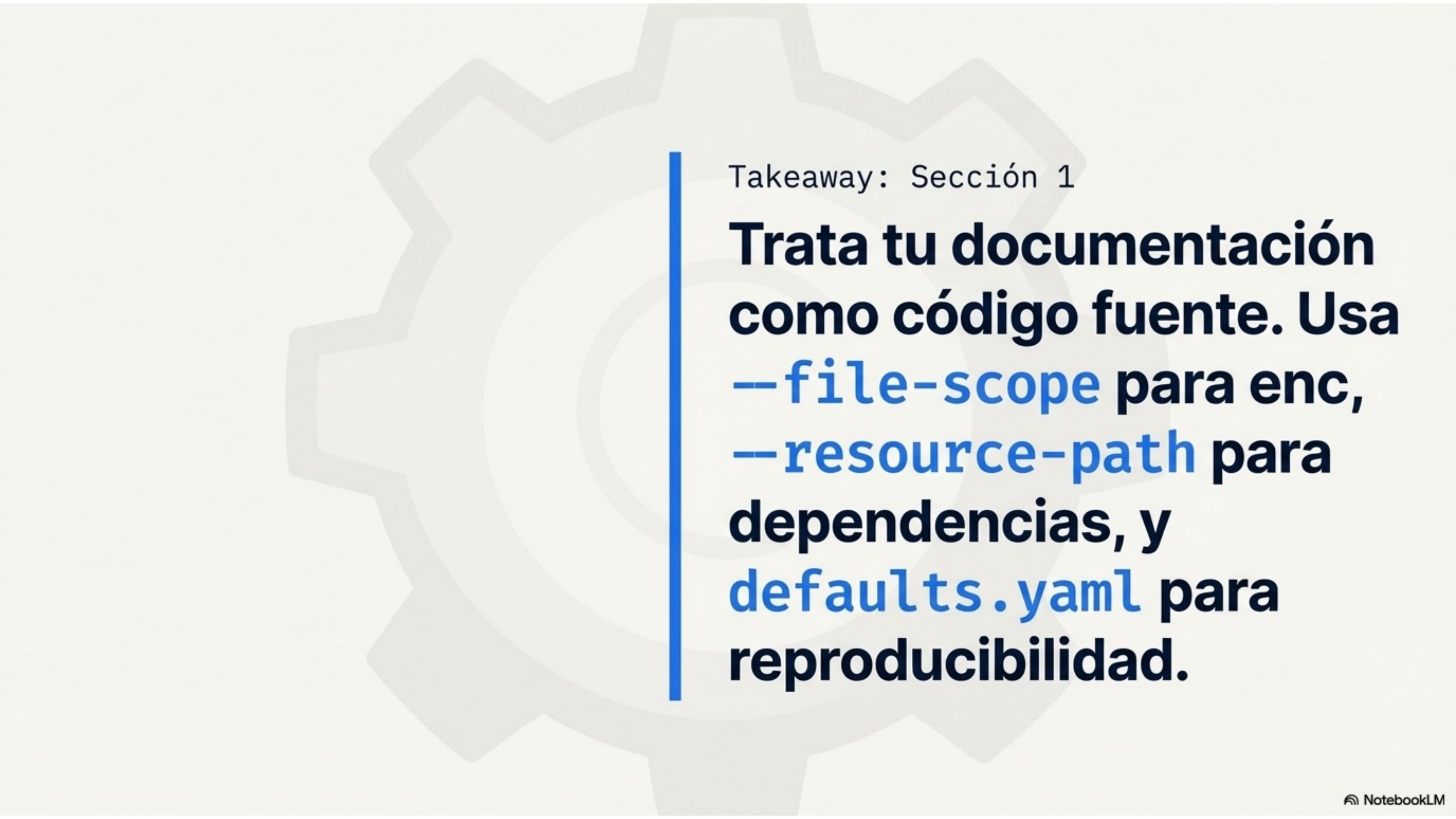
Mejor Práctica: defaults.yaml



```
# config/build.yaml
input-files:
  - src/01-intro.md
  - src/02-architecture.md

file-scope: true
resource-path: ['.', 'assets/img']
output-file: dist/documentation.pdf
```

Ventajas: Versionable en Git, interpolación de variables, y estructura jerárquica escalable.



Takeaway: Sección 1

Trata tu documentación como código fuente. Usa `--file-scope` para enc, `--resource-path` para dependencias, y `defaults.yaml` para reproducibilidad.

Selección del Motor PDF



pdflatex

Por defecto.
Alta compatibilidad histórica.
Requiere paquetes masivos de TeX Live.
Soporte UTF-8 limitado sin plantillas.



lua_latex

Estándar Moderno.
Soporte nativo UTF-8 y carga directa de fuentes del SO.
Permite creación de estándares de accesibilidad PDF/A y PDF/UA.

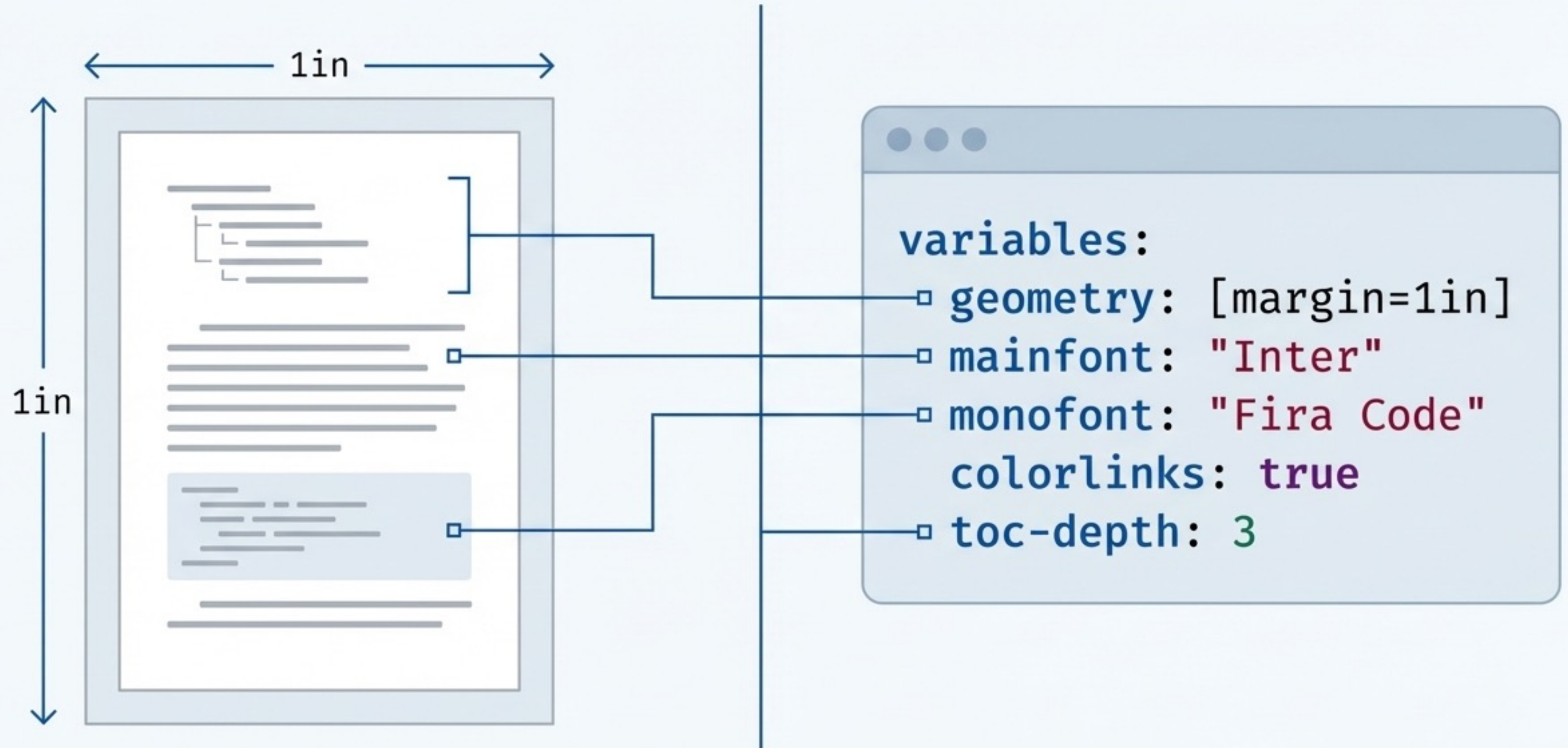


typst

Recomendado para CI/CD.
Motor en Rust de nueva generación.
Compilación ultra-rápida y configuración simple, ideal para pipelines automatizados.

Inyección de Variables de Plantilla

Control tipográfico mediante el bloque variables (-V) sin alterar la plantilla.



Renderizado de Bloques de Código

- **El Desafío:** El código debe ser legible, respetando la tabulación y los colores sintácticos.
- **El Motor Interno:** Pandoc usa definiciones de sintaxis KDE. Usa `--syntax-highlighting=idiomatic`.
- **Atributos Markdown:** Añadir clases permite numerar líneas y crear anclajes.

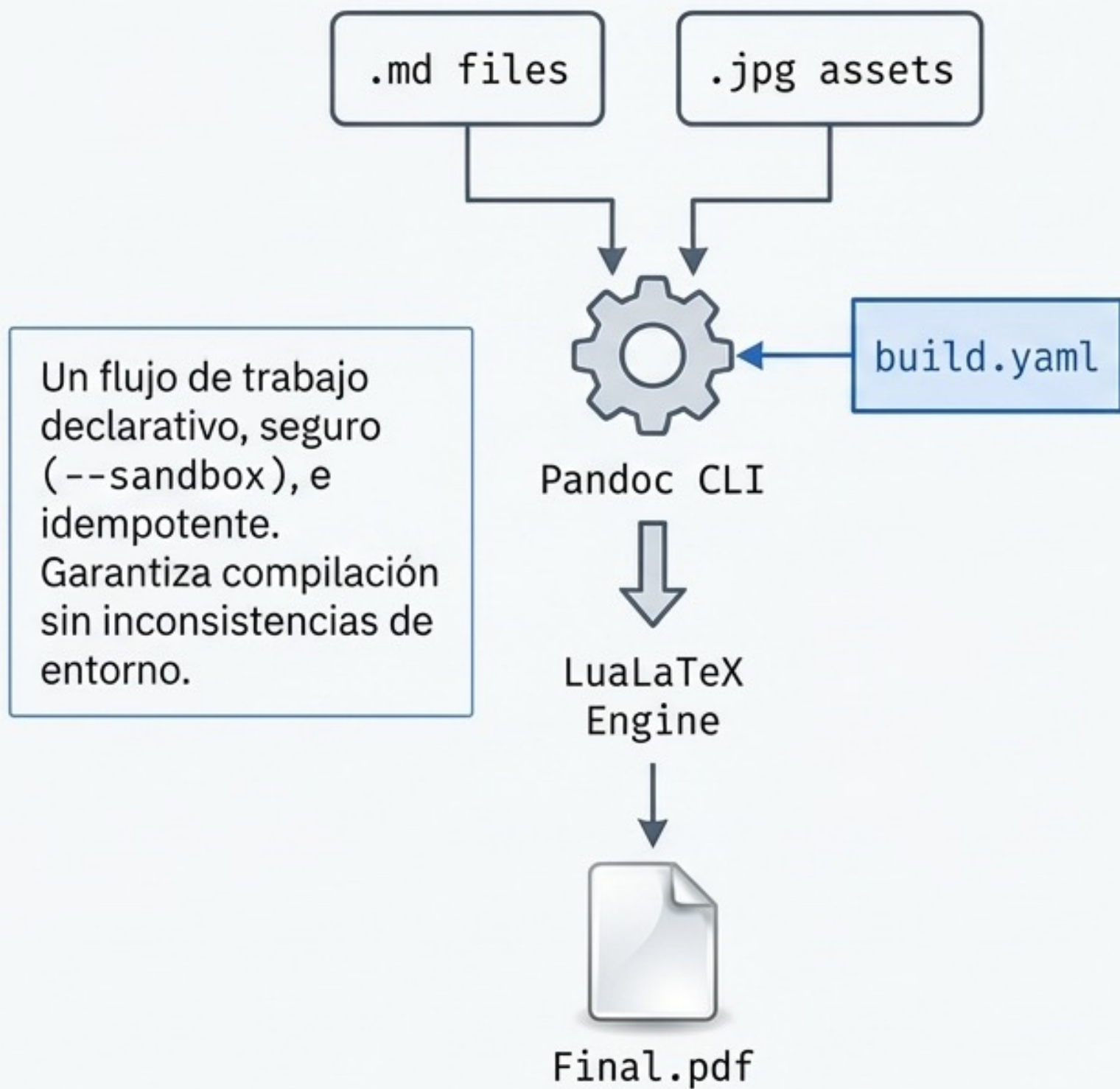
Sin Motor de Resaltado

```
def compile_pdf():  
    print("Building AST...")
```

Markdown con Atributos Extendidos

```
def compile_pdf():  
    print("Building AST...")
```


Ensamblaje del Pipeline de Producción



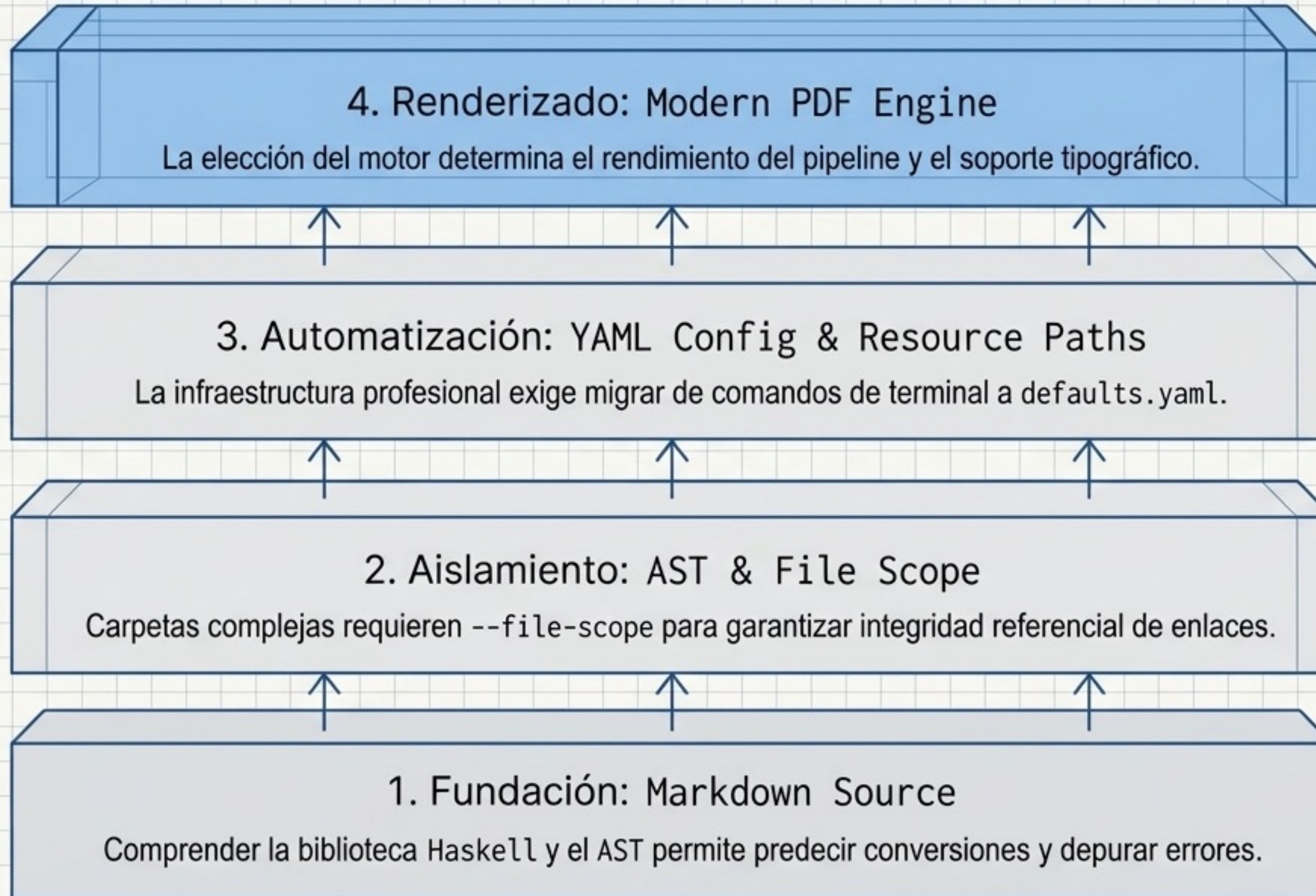
```
Bash
# CLI command reduces to:
pandoc --sandbox
      --defaults=config/build.yaml

YAML
# config/build.yaml snippet:
pdf-engine: lualatex
pdfstandard: a-4f # PDF/A compliance
syntax-highlighting: tango
number-sections: true
```

Takeaway: Sección 2

**Desacopla el contenido del diseño.
Confía la composición tipográfica
a motores modernos y controla el
diseño global fclon mediante
variables declarativas.**

Resumen General: El Modelo Mental



Referencias del Manual

Documentación oficial para profundizar en casos de uso avanzados.



Pandoc - User's Guide

Arquitectura base, manipulación de la biblioteca Haskell subyacente y el sistema de AST.



Specifying Formats & File Scope

Guía oficial sobre la concatenación segura de archivos múltiples para evitar colisiones.



PDF Engine Documentation

Comparativa de variables admitidas según el renderizador (LaTeX, Typst, wkhtmltopdf).



Defaults Files

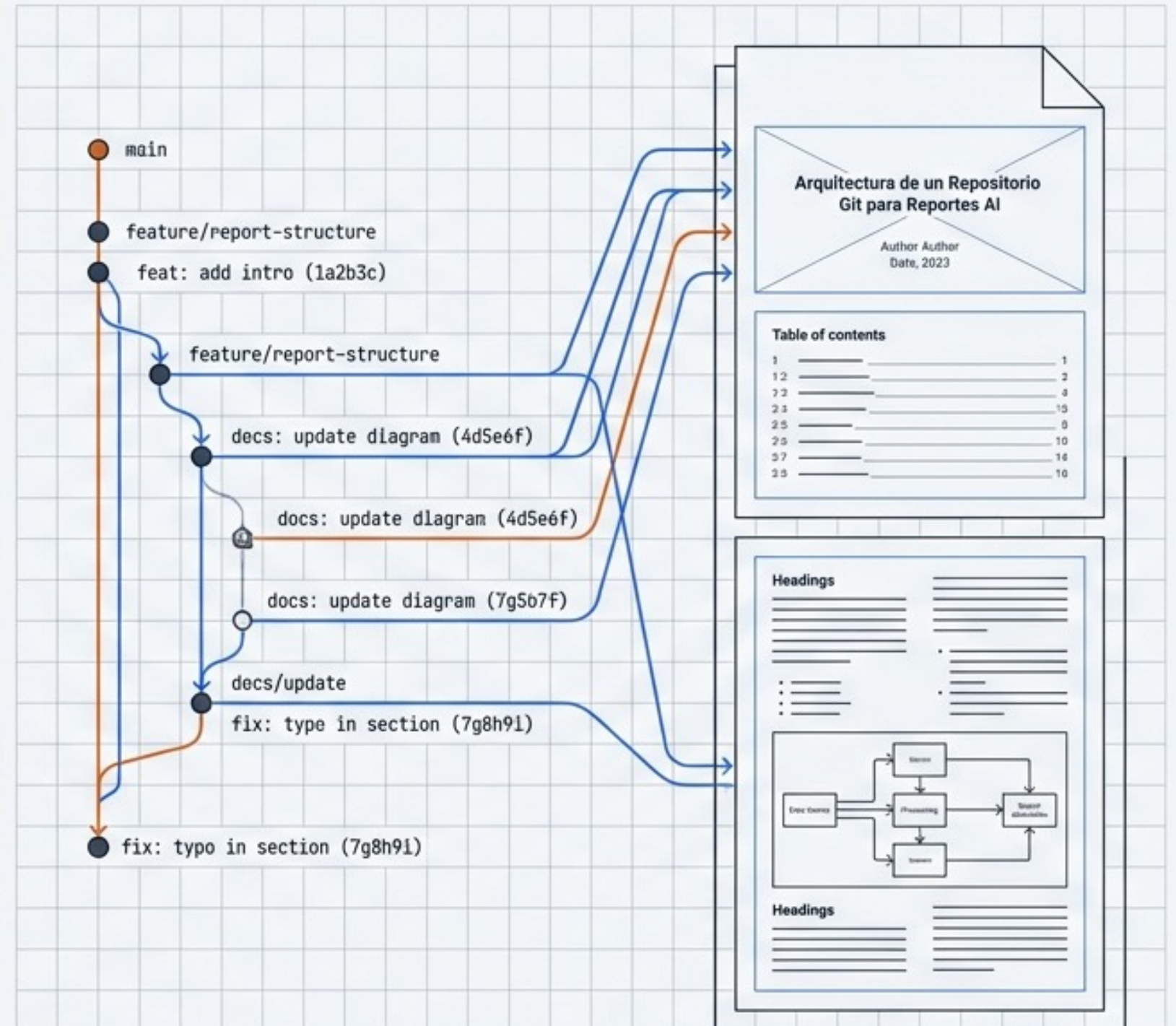
Sintaxis oficial YAML/JSON para la orquestación declarativa de pipelines.

¿Listo para compilar?

```
$ pandoc --defaults=master.yaml && echo "Build successful."
```

Arquitectura de un Repositorio Git para Reportes en Markdown

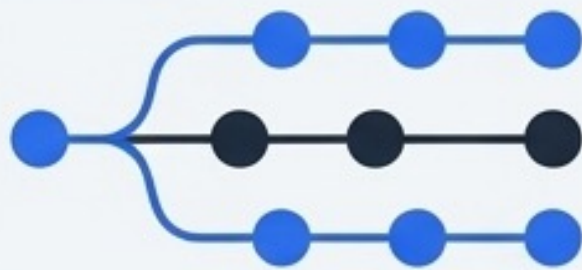
Guía de Docs-as-Code para Ingeniería de Software



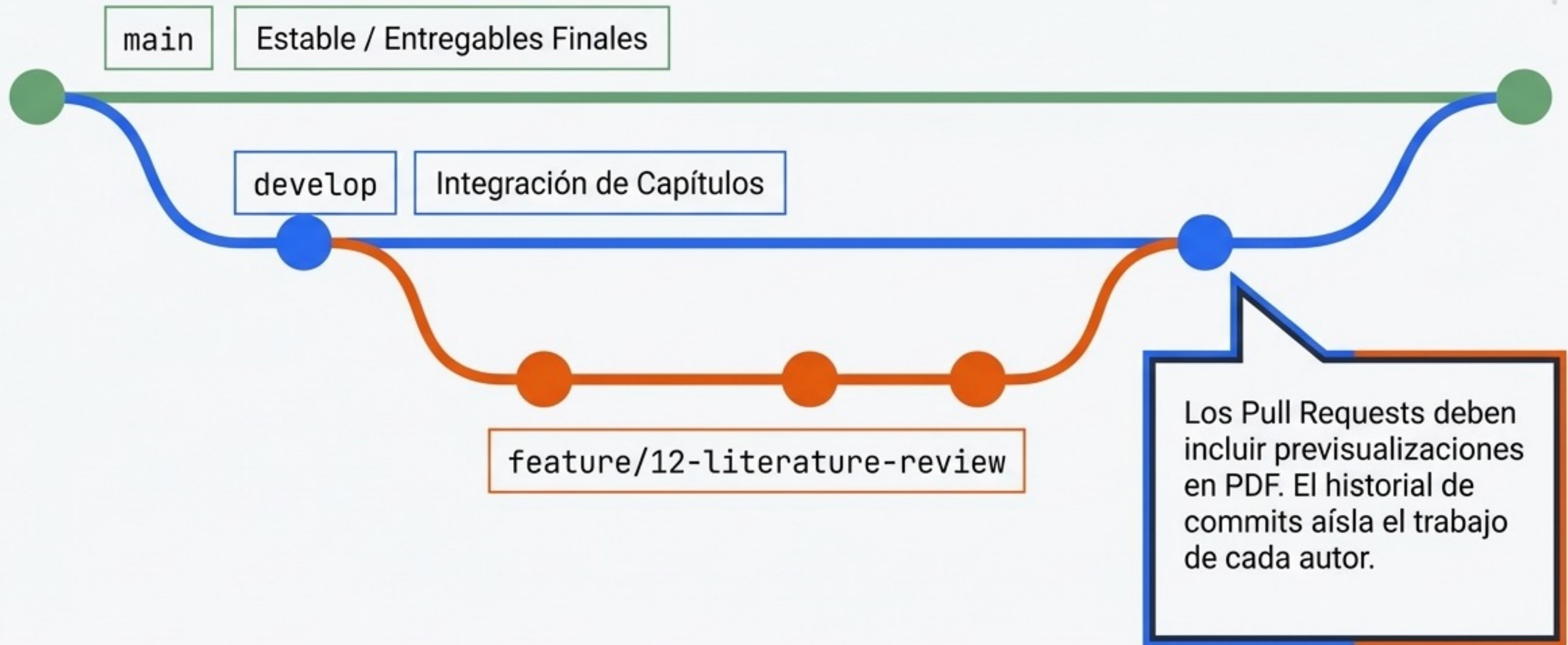
Agenda del Pipeline



El Paradigma Docs-as-Code

| | | |
|---------------------------------|---|---|
| |  |  |
| Control de Versiones | Caos de archivos:
<code>final_v3_definitivo.docx</code> | Commits rastreables y etiquetados. |
| Resolución de Conflictos | Sobrescritura manual y pérdida de datos. | Resolución controlada mediante Merge y Pull Requests. |
| Responsabilidad | Imposible auditar quién escribió qué. | Un autor principal asignado por cada versión en el historial. |
| Gráficos | Imágenes estáticas que requieren software externo. | Diagramas versionables generados mediante código. |

GitFlow para Documentación Colaborativa





**Trata tu reporte como software:
el control de versiones aísla el
trabajo y elimina la fricción
colaborativa.**

Arquitectura de Directorios y Nomenclatura

```
project-report-repo/  
├── report/  
│   ├── front-matter/  
│   ├── 01-abstract.md  
│   ├── 11-introduction.md  
│   ├── 12-literature-review.md  
│   ├── 99-bibliography.md  
│   ├── annexes/  
│   │   └── annex-a-raw-data.md  
│   └── assets/  
│       └── cover-background.pdf
```

Prefijos Numéricos (Kebab-case):

Sistema de orden estricto. 01-09 para front-matter, 10-89 para contenido principal, y 99 para el final. Esto garantiza la compilación secuencial.

Modularidad: Los anexos tienen su propia convención para aislar datos masivos.

Recursos: Aísla logos, portadas y diagramas generados.

Anatomía de un Archivo Markdown



Código Fuente

```
1  ```
2  title: Introduction
3  author: Alice Johnson
4  ---
5  # Introducción Principal -----
6  El análisis demuestra que... [@ref]
7
8  ## Contexto Secundario
9  Detalles del proyecto.
10 ```
```

Renderizado PDF

1. Introducción Principal

El análisis demuestra que... [1]

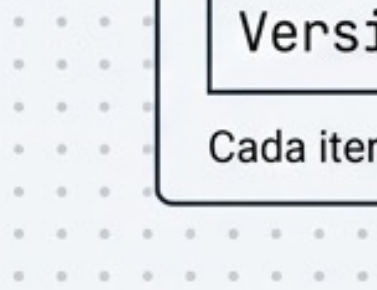
1.1 Contexto Secundario

Detalles del proyecto.

Control de Versiones: 10-version-history.md

| Versión | Fecha | Autor Principal |
|-------------|------------|-----------------|
| Version 1.0 | 2025-12-15 | Alice Johnson |

Cada iteración documenta explícitamente a un Autor Principal.





La modularidad es clave:
divide y vencerás. Archivos
pequeños y ordenados
numéricamente **evitan**
conflictos de integración.

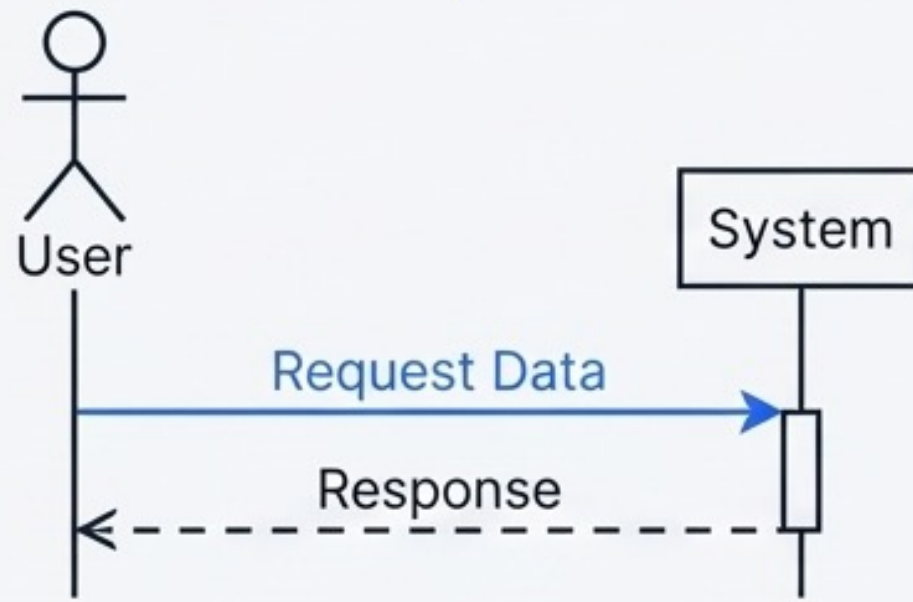


Diagramas como Código: PlantUML y Mermaid

PlantUML (Diagramas de Secuencia)

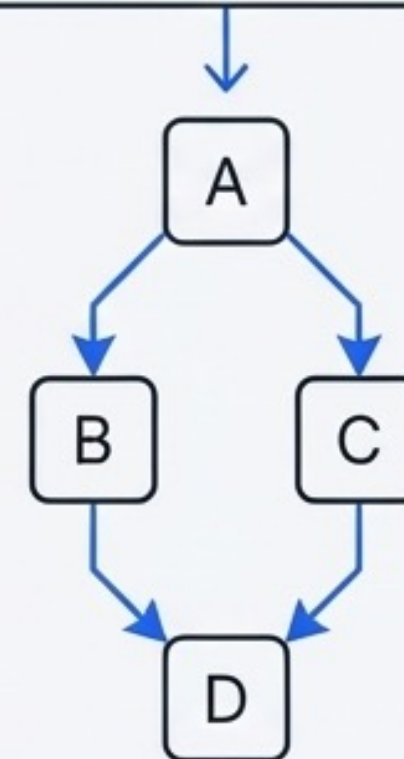
```
@startuml
actor User

User -> System: Request Data
System --> User: Response
@enduml
```

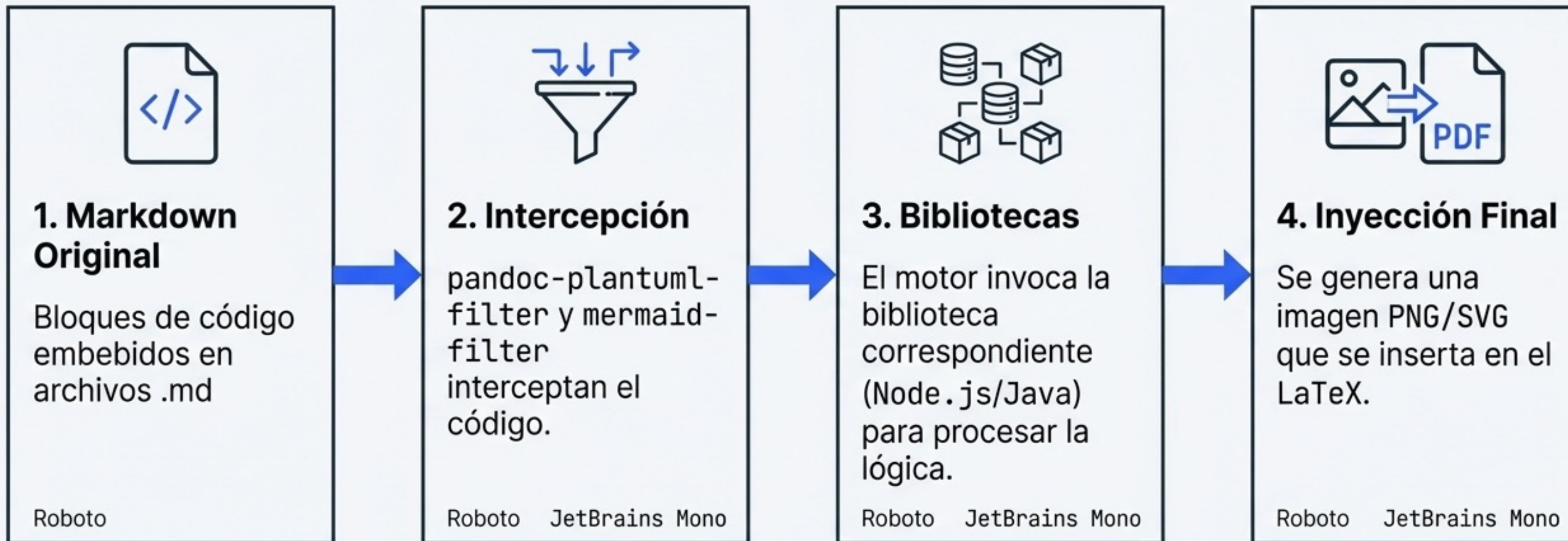
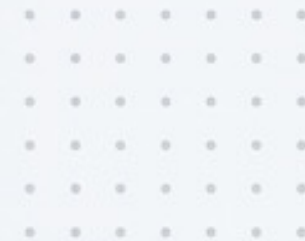


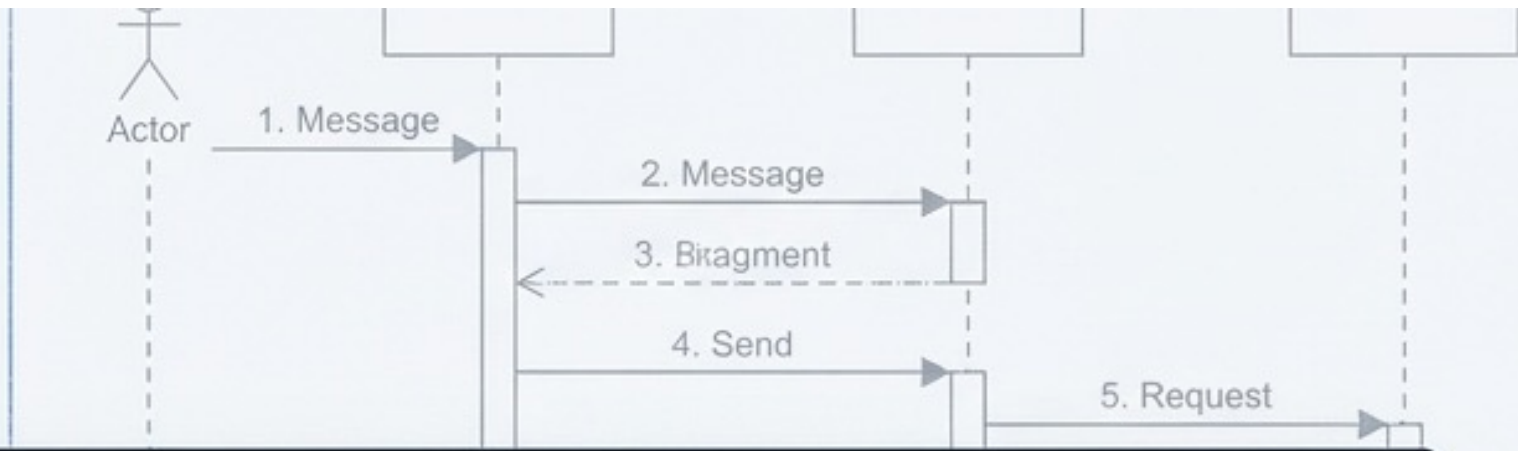
Mermaid (Diagramas de Flujo)

```
mermaid
graph TD;
    A-->B;
    A-->C;
    B-->D;
    C-->D;
```



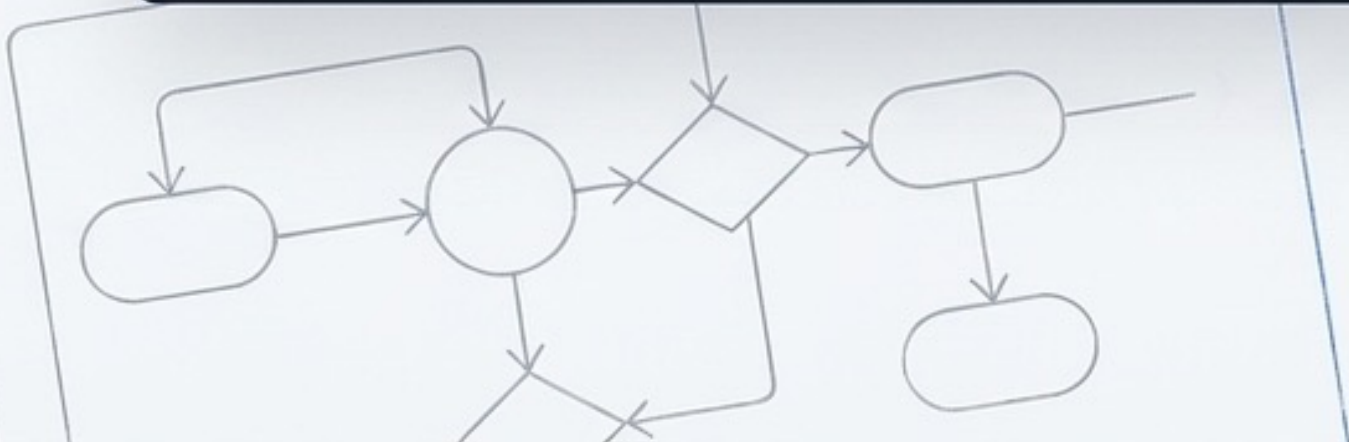
Integración de Filtros de Visualización





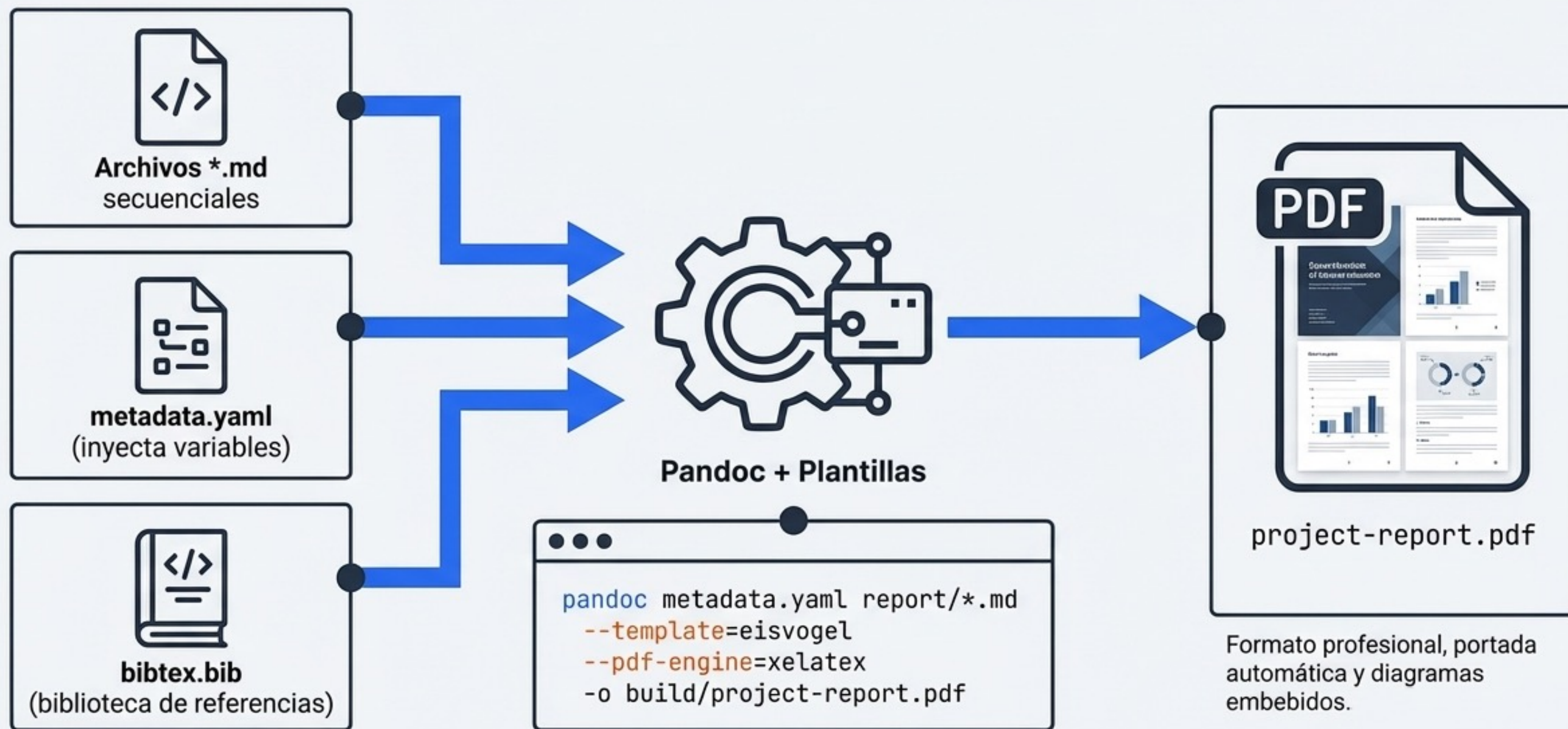
UML sequence diagram showing an Actor interacting with three lifelines. The sequence of messages is: 1. Message (solid arrow from Actor to lifeline 1), 2. Message (solid arrow from lifeline 1 to lifeline 2), 3. Fragment (dashed arrow from lifeline 2 to lifeline 1), 4. Send (solid arrow from lifeline 1 to lifeline 3), and 5. Request (solid arrow from lifeline 3 to lifeline 2).

Mantén los gráficos versionables:
usar diagramas como código asegura
que las imágenes evolucionen de
forma sincronizada con el texto.



UML activity diagram showing a flow starting from a start node (solid circle) to an activity node (rounded rectangle), then to a decision node (diamond). From the decision node, the flow splits into two paths: one leading to an activity node and another leading to a final node (solid circle).

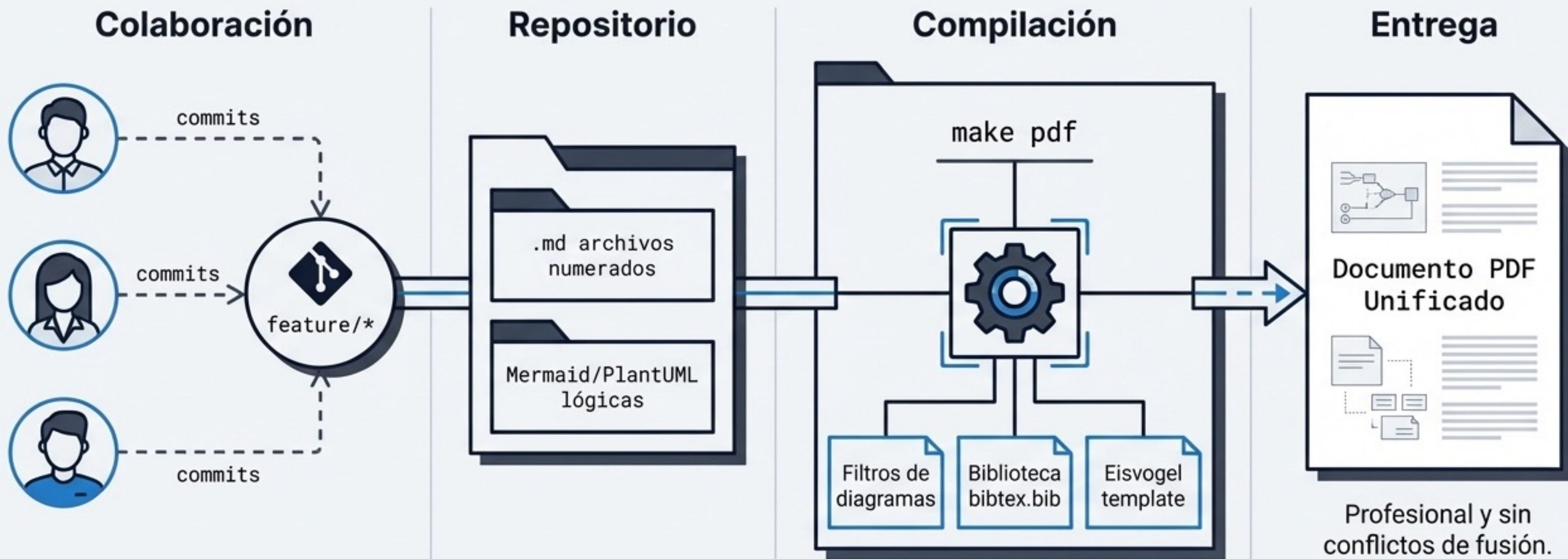
El Motor de Construcción: Pandoc + Eisvogel





Automatiza la presentación:
separa el contenido (**Markdown**)
del diseño (**Eisvogel**)
centralizando la compilación en
un archivo **Makefile** robusto.

Resumen General de la Arquitectura



Tecnologías Base

GitFlow, Pandoc, Eisevogel LaTeX Template

Filtros de Visualización

PlantUML, Mermaid.js

Construye tu documentación con la misma ingeniería que tu código.

PREGRADO

Ingeniería de Software

Escuela de Ingeniería de Sistemas y Computación | Facultad de Ingeniería



UPC

Universidad Peruana
de Ciencias Aplicadas

Prolongación Primavera 2390,
Monterrico, Santiago de Surco

Lima 33 - Perú

T 511 313 3333

<https://www.upc.edu.pe>

CONVENIOS INTERNACIONALES

La UPC ha sido considerada la universidad más internacional del Perú desde el año 2014 y cuenta con más de 200 convenios internacionales con prestigiosas universidades alrededor del mundo.



exígete, innova

WASC

Senior College and
University Commission

Universidad Peruana de Ciencias Aplicadas is accredited by the WASC Senior College and
University Commission (WSCUC), 985 Atlantic Avenue, Suite 100, Alameda, CA 94501, 510.748.9001

